Université Cheikh Anta Diop de Dakar
Faculté des Sciences et Techniques
Département de Mathématiques et Informatique, section Informatique

RAPPORT DE PROJET · MASTER 2 BUSINESS INTELLIGENCE

Conception d'une Modern Data Platform avec architecture Médaillon et application Text-to-SQL basée sur l'IA

Wide World Importers · SQL Server, DLT, dbt, Dagster, Snowflake, Streamlit

Réalisé par
Ibrahima Lang Diop

Encadrant
Dr. Jean Gane Sarr

Année universitaire
2025–2026

Résumé

Ce projet met en place une plateforme de données moderne à partir de la base transactionnelle *Wide World Importers* de Microsoft. Les données de SQL Server sont extraites avec DLT et chargées dans Snowflake, où elles sont organisées en trois couches : Bronze pour les données brutes, Silver pour les données nettoyées et Gold pour un modèle en étoile destiné à l'analyse. Les transformations sont écrites avec dbt et vérifiées par dix-huit tests de qualité ; l'ensemble est orchestré par Dagster, qui exécute la chaîne complète en sept minutes et demie et la planifie chaque nuit.

Une application Streamlit permet ensuite d'interroger la couche Gold en français. La question est traduite en SQL par un modèle de langage, la requête est contrôlée puis exécutée, et le résultat est présenté sous forme de graphique et de tableau. Le compte d'essai Snowflake ne donnant pas accès à la fonction Cortex `COMPLETE`, un modèle local servi par Ollama prend le relais, sans modification du reste de l'architecture.

Mots-clés : Modern Data Platform, architecture Médaillon, ELT, Snowflake, dbt, Dagster, Text-to-SQL.

Abstract

This project builds a modern data platform on top of Microsoft's *Wide World Importers* transactional database. Data is extracted from SQL Server with DLT and loaded into Snowflake, where it is organised into three layers: Bronze for raw data, Silver for cleaned data and Gold for a star schema designed for analysis. Transformations are written with dbt and checked by eighteen data quality tests; Dagster orchestrates the whole chain, which runs in about seven and a half minutes and is scheduled every night.

A Streamlit application lets users query the Gold layer in French. Each question is translated into SQL by a language model, the query is checked and executed, and the result is shown as a chart and a table. Since the Snowflake trial account does not allow the Cortex `COMPLETE` function, a local model served by Ollama generates the SQL instead, with no change to the rest of the architecture.

Keywords: Modern Data Platform, Medallion architecture, ELT, Snowflake, dbt, Dagster, Text-to-SQL.

Table des matières

Résumé i

Abstract ii

Introduction générale 1

Problématique 1

Objectifs 1

Démarche et organisation du rapport 2

1 Cadre et choix techniques 3

1.1 De l'entrepôt de données à la Modern Data Platform 3

1.2 L'architecture Médaillon 3

1.3 La modélisation dimensionnelle 4

1.4 Le Text-to-SQL avec un modèle de langage 4

1.5 Outils retenus 4

2 Analyse et conception 6

2.1 Le jeu de données Wide World Importers 6

2.2 Besoins d'analyse 6

2.3 Architecture générale 7

2.4 Tables sources retenues 7

2.5 Modèle de la couche Gold 7

2.6 Environnements de développement et de production 8

3 Mise en œuvre de la plateforme 10

3.1 Initialisation du projet 10

3.2 Ingestion de la couche Bronze avec DLT 12

3.3 Transformations Silver et Gold avec dbt 14

3.3.1 Déclaration des sources 14

3.3.2 Placement des modèles dans les schémas 15

3.3.3 Couche Silver 15

3.3.4 Couche Gold 16

3.3.5 Tests de qualité . . . 17
3.4 Orchestration avec Dagster . . . 18
3.4.1 Déclaration des composants . . . 18
3.4.2 Planification . . . 19
3.4.3 Exécution . . . 20
3.5 Passage à Snowflake . . . 23
3.5.1 Droits et propriété des schémas . . . 23
3.5.2 Profil dbt . . . 24
3.6 Application Text-to-SQL WWI Insight . . . 26
3.6.1 Organisation du code . . . 26
3.6.2 Consigne transmise au modèle . . . 27
3.6.3 Choix du moteur de génération . . . 28
3.6.4 Contrôle des requêtes . . . 28
3.6.5 Sessions et dossiers . . . 29
3.6.6 Une question traitée de bout en bout . . . 31

4 Résultats et discussion **34**
4.1 Volumes et qualité des données . . . 34
4.2 Performances . . . 34
4.3 Réponses aux questions de la consigne . . . 35
4.3.1 Les dix meilleurs clients . . . 35
4.3.2 Les ventes par mois . . . 36
4.3.3 Les produits les plus rentables . . . 37
4.4 Évaluation du Text-to-SQL . . . 37
4.5 Difficultés rencontrées . . . 37
4.6 Limites . . . 38

Conclusion générale **39**
Perspectives . . . 39

Annexes **41**
A. Commandes principales . . . 41
B. Variables d'environnement . . . 41
C. Exécution d'une question dans l'application . . . 42
D. Lancement de l'application . . . 42

Table des figures

2.1 Architecture de la plateforme . . . 7
2.2 Schéma en étoile de la couche Gold . . . 8

3.1 Création du projet avec `create-dagster` . . . 10
3.2 Ajout des dépendances et génération du composant DLT . . . 11
3.3 Arborescence du projet . . . 12
3.4 Avertissements de DLT sur les colonnes `geography` lors des premiers chargements 13
3.5 Fichier `.dlt/secrets.toml` (valeurs masquées) . . . 13
3.6 Lignage des modèles dbt, des sources Bronze aux tables Gold . . . 18
3.7 Actifs et définitions chargés, vérifiés avec `dg list defs` . . . 19
3.8 Lignage des actifs dans Dagster, de l'ingestion DLT aux tables Gold . . . 21
3.9 Étape d'ingestion DLT terminée . . . 22
3.10 Étape dbt terminée, avec les contrôles de qualité . . . 22
3.11 Exécution complète du job `pipeline_complet` . . . 23
3.12 Tables du schéma `GOLD` dans Snowsight, propriété de `PFPB_ROLE` . . . 24
3.13 Vérification de la connexion à Snowflake avec `dbt debug` . . . 25
3.14 Nombre de lignes par table dans les trois schémas . . . 26
3.15 Aperçu de la table `FCT_SALES` dans Snowsight . . . 26
3.16 Page d'accueil connectée à Snowflake . . . 27
3.17 Requête d'écriture refusée par l'application . . . 29
3.18 Dossier vide et questions d'exemple . . . 31
3.19 Réponse à une question libre, SQL généré par Ollama . . . 32
3.20 Requête générée, affichée dans l'onglet SQL . . . 32
3.21 Page « À propos » . . . 33

4.1 Les dix meilleurs clients par chiffre d'affaires (exécution locale DuckDB) . . . 35
4.2 Requête exécutée pour la première question . . . 36
4.3 Chiffre d'affaires par mois (exécution locale DuckDB) . . . 36
4.4 Les produits qui génèrent le plus de bénéfice (exécution locale DuckDB) . . . 37

Liste des codes

3.1 Déclaration de la source DLT (defs/mssql_ingestion/loads.py) 12
3.2 Choix de la destination et définition du pipeline (loads.py) 13
3.3 Sources Bronze déclarées dans dbt (models/_sources.yml) 14
3.4 Surcharge du nommage des schémas (macros/generate_schema_name.sql) . 15
3.5 Vue Silver des lignes de facture (models/staging/stg_invoice_lines.sql) 15
3.6 Table de faits des ventes (models/marts/fct_sales.sql) 16
3.7 Documentation et tests de fct_sales (models/marts/_marts.yml) 17
3.8 Déclaration des composants DLT et dbt (fichiers defs.yaml) 18
3.9 Job complet et planification quotidienne (defs/schedules.py) 19
3.10 Préparation du compte Snowflake (feuille de calcul Snowsight) 23
3.11 Profil dbt à deux cibles (defs/transform/profiles.yml) 24
3.12 Consigne envoyée au modèle de langage (presentation_app/warehouse.py) . 27
3.13 Sélection du moteur de génération (warehouse.py) 28
3.14 Contrôle d'une requête avant exécution (warehouse.py) 29
3.15 Création et restauration d'une session (users_db.py et session.py) 30

Introduction générale

Une entreprise de distribution enregistre chaque jour des commandes, des factures, des mouvements de stock et des paiements. Ces données sont stockées dans des bases transactionnelles conçues pour enregistrer rapidement des opérations, et non pour répondre à des questions d'analyse. Savoir quels clients rapportent le plus, comment évoluent les ventes d'un mois à l'autre ou quels produits dégagent la meilleure marge suppose de rassembler les données, de les nettoyer et de les réorganiser. En pratique, ce travail repose souvent sur une équipe technique, et un responsable commercial doit attendre qu'un analyste écrive la requête SQL dont il a besoin.

Deux évolutions récentes changent cette situation. D'une part, les plateformes de données en nuage comme Snowflake séparent le stockage du calcul et permettent de charger les données brutes avant de les transformer sur place ; c'est l'approche ELT (*Extract, Load, Transform*). Des outils spécialisés couvrent chaque étape : DLT pour l'ingestion, dbt pour les transformations, Dagster pour l'orchestration. D'autre part, les grands modèles de langage savent désormais produire une requête SQL à partir d'une question formulée en langage naturel, ce qui ouvre l'accès aux données à des utilisateurs qui ne connaissent pas SQL.

Problématique

Le projet répond à la question suivante : comment concevoir une plateforme qui automatise l'ingestion, la transformation et la modélisation des données d'une base transactionnelle, puis qui permet de les interroger en français, de façon fiable et sans connaissance de SQL ?

Objectifs

L'objectif est de construire une chaîne complète sur la base *Wide World Importers* de Microsoft. Concrètement, il s'agit :

- d'extraire automatiquement les données de SQL Server et de les charger dans Snowflake avec DLT ;
- d'organiser l'entrepôt selon une architecture Médaillon, en trois couches Bronze, Silver et Gold ;
- d'écrire les transformations avec dbt, en les documentant et en les testant ;

- d’orchestrer et de planifier l’ensemble avec Dagster ;
- de développer une application Streamlit qui traduit une question en SQL avec un modèle de langage et affiche le résultat.

Démarche et organisation du rapport

Nous avons d’abord développé toute la chaîne sur une base DuckDB locale, gratuite et sans configuration, avant de basculer vers Snowflake en changeant uniquement des variables d’environnement. Ce choix a permis de corriger rapidement les modèles dbt et l’application sans consommer de crédits.

Le chapitre 1 présente les notions et les outils sur lesquels repose le projet. Le chapitre 2 décrit le jeu de données, les besoins d’analyse et l’architecture retenue. Le chapitre 3 détaille la mise en œuvre de chaque brique, en s’appuyant sur des extraits du code du projet. Le chapitre 4 présente les résultats obtenus, les difficultés rencontrées et les limites de la solution. La conclusion dresse le bilan du projet et propose des suites possibles.

CHAPITRE 1

Cadre et choix techniques

Ce chapitre présente les notions sur lesquelles repose le projet, puis les outils retenus pour chaque étape de la chaîne.

1.1 De l'entrepôt de données à la Modern Data Platform

L'entrepôt de données classique repose sur une chaîne ETL : les données sont extraites des systèmes sources, transformées sur un serveur intermédiaire, puis chargées dans l'entrepôt sous leur forme finale. Les transformations sont écrites dans un outil dédié et s'exécutent avant le chargement, ce qui rend toute correction coûteuse : il faut relancer l'extraction pour reconstruire une table.

Les entrepôts en nuage ont inversé cet ordre. Le stockage y est bon marché et la puissance de calcul se loue à la demande ; il devient donc raisonnable de charger d'abord les données brutes, puis de les transformer dans l'entrepôt lui-même, en SQL. On parle alors d'ELT. Une *Modern Data Platform* assemble autour de cet entrepôt des outils spécialisés, chacun responsable d'une étape : un outil d'ingestion, un outil de transformation, un orchestrateur et des applications de restitution. Chaque brique peut être remplacée sans reconstruire les autres, ce que nous avons d'ailleurs exploité en passant de DuckDB à Snowflake.

1.2 L'architecture Médaillon

L'architecture Médaillon organise l'entrepôt en trois couches successives, chacune lisant la précédente.

- **Bronze** contient les données telles qu'elles arrivent de la source, sans règle métier. Elle sert de point de reprise : si une transformation est erronée, on la corrige et on la rejoue sans solliciter à nouveau SQL Server.
- **Silver** contient des données nettoyées et typées : colonnes renommées, dates converties, lignes invalides écartées, jointures techniques effectuées.
- **Gold** contient des tables conçues pour l'analyse, ici un schéma en étoile. C'est la seule couche exposée aux utilisateurs et à l'application.

Cette séparation rend chaque étape lisible et testable isolément. Elle permet aussi de limiter les droits : l'application n'a besoin que de lire la couche Gold.

1.3 La modélisation dimensionnelle

La couche Gold suit la modélisation dimensionnelle de Ralph Kimball. Une **table de faits** enregistre des événements mesurables, par exemple une ligne de commande avec sa quantité et son montant. Les **tables de dimensions** décrivent le contexte de ces événements : le client, le produit, le vendeur, la date. Chaque table de faits a un **grain**, c'est-à-dire ce que représente exactement une ligne ; le fixer dès la conception évite les doubles comptes lors des agrégations.

Ce modèle convient particulièrement au Text-to-SQL : un schéma en étoile comporte peu de tables, des noms de colonnes explicites et des jointures simples, ce qui réduit les erreurs du modèle de langage.

1.4 Le Text-to-SQL avec un modèle de langage

Un système Text-to-SQL traduit une question en langage naturel en requête SQL. Avec un grand modèle de langage, la traduction repose sur une consigne (*prompt*) qui contient la description du schéma, quelques règles métier et la question de l'utilisateur. Le modèle n'a pas accès aux données : il ne voit que les noms des tables et des colonnes.

Cette approche comporte trois limites, que l'application doit prendre en compte :

- le modèle peut produire une requête syntaxiquement correcte mais fausse sur le plan métier, par exemple en sommant un montant TTC au lieu du montant hors taxes ;
- il peut produire une requête qui modifie les données si la question s'y prête ;
- sa réponse n'est pas toujours limitée au SQL : il ajoute parfois une explication ou un bloc de code Markdown.

La première limite se réduit en donnant au modèle les définitions métier attendues ; les deux autres imposent de contrôler chaque requête avant de l'exécuter.

1.5 Outils retenus

Le tableau 1.1 résume le rôle de chaque outil dans le projet.

Tableau 1.1 Outils de la plateforme et rôle de chacun

Brique	Outil	Rôle dans le projet
Source	SQL Server	Héberge la base transactionnelle *WideWorldImporters*
Ingestion	DLT	Extrait dix tables et les charge dans la couche Bronze
Entrepôt	Snowflake	Stocke les trois couches ; DuckDB en local pendant le développemer
Transformation	dbt	Construit les couches Silver et Gold, exécute les tests
Orchestration	Dagster	Enchaîne DLT et dbt, affiche le lignage, planifie l'exécution
Génération du SQL	Cortex, Ollama	Traduit une question en SQL (Cortex sur Snowflake, sinon modèle loc
Restitution	Streamlit	Application WWI Insight : questions, résultats, dossiers

Ces outils ont un point commun : leur configuration est du code (Python, SQL ou YAML) versionné avec Git. L'ensemble du projet tient dans un seul dépôt et se reconstruit avec quelques commandes, rappelées en annexe.

CHAPITRE 2

Analyse et conception

2.1 Le jeu de données Wide World Importers

Wide World Importers est une base d'exemple publiée par Microsoft. Elle décrit un grossiste fictif qui importe et revend des articles de nouveauté et d'emballage (rubans adhésifs, papier bulle, cartons, figurines) à des magasins de jouets et à des particuliers aux États-Unis. La base transactionnelle *WideWorldImporters* est organisée en plusieurs schémas, dont trois nous intéressent :

- `Sales` : clients, commandes, lignes de commande, factures et lignes de facture ;
- `Warehouse` : articles en stock et leurs caractéristiques ;
- `Application` : personnes (dont les vendeurs) et référentiel géographique.

Les commandes couvrent la période de janvier 2013 à mai 2016, soit 41 mois. La base contient 73 595 commandes, 231 412 lignes de commande, 663 clients et 227 articles.

2.2 Besoins d'analyse

La consigne cite trois questions types, que l'application doit traiter :

1. Quels sont les dix meilleurs clients par chiffre d'affaires ?
2. Quel est le montant total des ventes par mois ?
3. Quels produits génèrent le plus de bénéfices ?

Elles imposent trois mesures : le chiffre d'affaires hors taxes, le bénéfice et un découpage par mois. Nous y avons ajouté des questions voisines, utiles pour un responsable commercial : le chiffre d'affaires par année et par vendeur, les produits les plus vendus en quantité, le bénéfice par année et le panier moyen par commande.

Une difficulté apparaît dès la troisième question : le bénéfice n'existe pas dans les commandes. Il n'est calculé qu'au moment de la facturation, dans la colonne `LineProfit` de la table `Sales.InvoiceLines`. Le modèle Gold doit donc comporter deux tables de faits, l'une pour les commandes, l'autre pour les factures.

2.3 Architecture générale

La figure 2.1 présente l'architecture retenue. DLT extrait les données de SQL Server et les charge dans le schéma `BRONZE` de Snowflake. dbt construit ensuite les vues du schéma `SILVER` et les tables du schéma `GOLD`. Dagster orchestre ces deux étapes et les planifie. L'application Streamlit lit uniquement la couche Gold ; elle délègue la génération du SQL à Snowflake Cortex ou, à défaut, à un modèle local servi par Ollama.

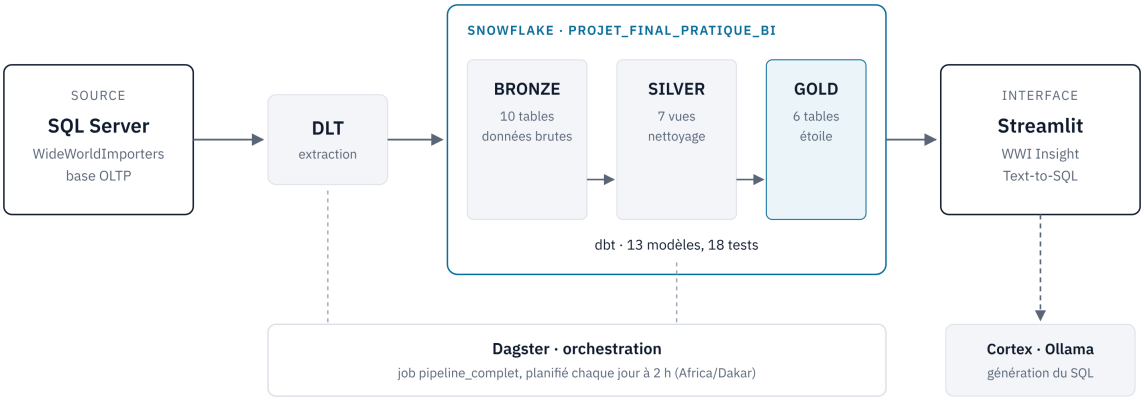


Figure 2.1 Architecture de la plateforme

2.4 Tables sources retenues

Nous n'avons chargé que les tables nécessaires aux analyses visées. Le tableau 2.1 précise l'usage de chacune.

Tableau 2.1 Tables extraites de *WideWorldImporters*

Table SQL Server	Usage dans le modèle
`Sales.Customers`	Dimension client et ville de livraison
`Sales.Orders`	En-tête des commandes : date, client, vendeur
`Sales.OrderLines`	Lignes de commande : article, quantité, prix
`Sales.Invoices`	En-tête des factures : date de facturation
`Sales.InvoiceLines`	Lignes de facture : montant facturé et bénéfice
`Warehouse.StockItems`	Dimension produit
`Application.People`	Dimension vendeur (`IsSalesperson = 1`)
`Application.Cities`	Ville de livraison des clients
`Application.StateProvinces`	Référentiel géographique (chargé pour des analyses futures)
`Application.Countries`	Référentiel géographique (chargé pour des analyses futures)

2.5 Modèle de la couche Gold

La couche Gold suit un schéma en étoile à deux tables de faits, représenté par la figure 2.2.

- `fct_sales` a pour grain la **ligne de commande**. Elle porte la quantité, le prix unitaire et le chiffre d'affaires hors taxes et toutes taxes comprises.
- `fct_invoice_lines` a pour grain la **ligne de facture**. Elle porte le montant facturé et le bénéfice.

Les deux tables partagent les dimensions `dim_customers`, `dim_products` et `dim_salespeople`. La dimension `dim_date` couvre chaque jour de 2013 à 2016 ; elle se joint aux faits sur la date de commande ou de facturation.

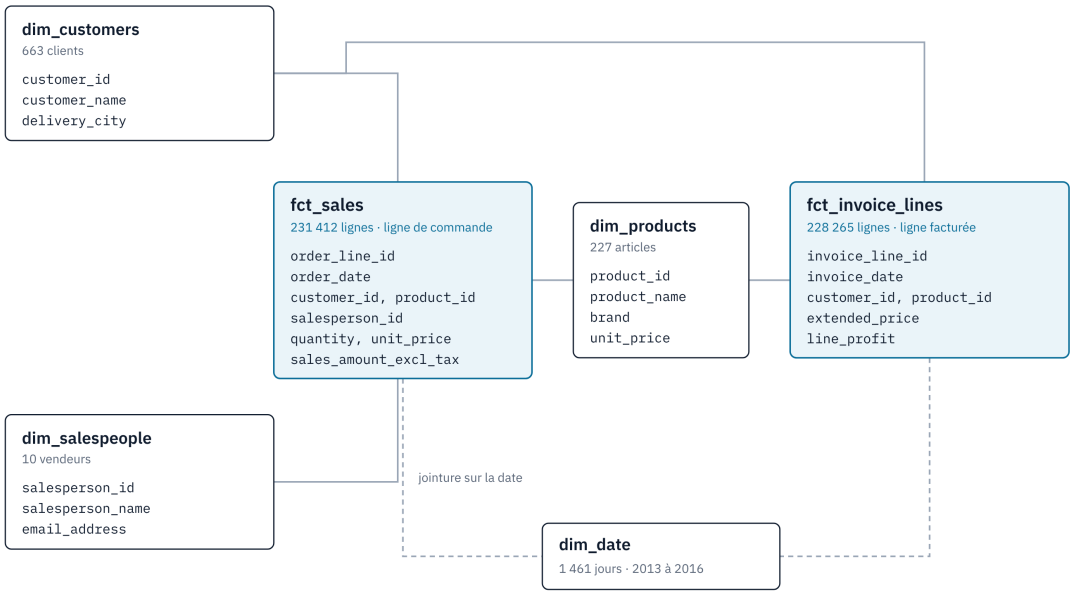


Figure 2.2 Schéma en étoile de la couche Gold

Le chiffre d'affaires est défini comme la somme de `quantity × unit_price`, hors taxes. Nous avons écrit cette définition dans la documentation dbt et dans la consigne transmise au modèle de langage, pour que l'application et un analyste obtiennent le même chiffre.

2.6 Environnements de développement et de production

Le même code s'exécute dans deux environnements. En développement, DLT et dbt écrivent dans un fichier DuckDB local ; en production, ils écrivent dans Snowflake. Le choix se fait uniquement par des variables d'environnement, regroupées dans le tableau 2.2.

Tableau 2.2 Variables d'environnement qui sélectionnent la destination

Variable	Développement (DuckDB)	Production (Snowflake)
`DLT_DESTINATION`	`duckdb`	`snowflake`
`DBT_TARGET`	`dev`	`prod`
`DUCKDB_PATH`	chemin du fichier local	non utilisée
`SF_ACCOUNT`, …	non utilisées	identifiants Snowflake

Ce découplage a deux avantages : il évite de consommer des crédits Snowflake pendant la mise au point, et il permet de tester une modification en local avant de la déployer.

CHAPITRE 3

Mise en œuvre de la plateforme

Ce chapitre suit l'ordre de la chaîne : initialisation du projet, ingestion, transformations, orchestration, passage à Snowflake, puis application. Les extraits de code sont tirés du dépôt ; les numéros de ligne cités renvoient aux extraits tels qu'ils sont reproduits.

3.1 Initialisation du projet

Le projet a été créé avec `create-dagster`, l'outil d'initialisation de Dagster, qui génère un projet Python géré par `uv` (figure 3.1). Les dépendances ont ensuite été ajoutées avec `uv add` : `dlt[snowflake,duckdb,mssql]`, `pyodbc` et `dagster-dlt` pour l'ingestion, `dagster-dbt`, `dbt-duckdb` et `dbt-snowflake` pour les transformations, `streamlit` et `bcrypt` pour l'application.

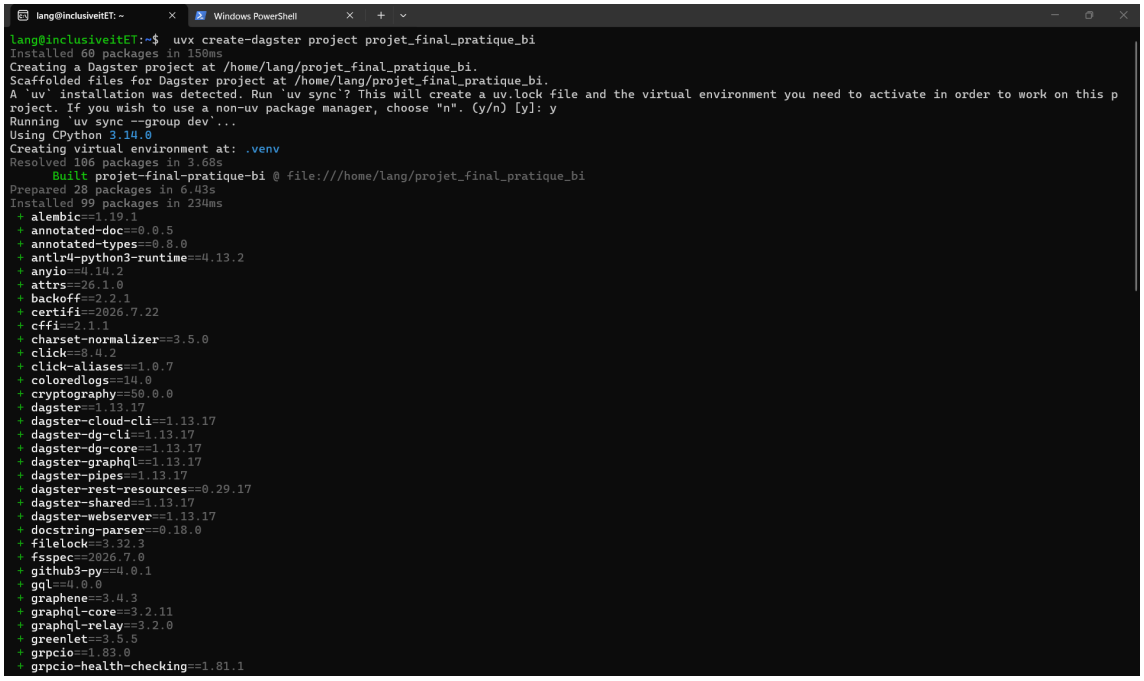


Figure 3.1 Création du projet avec `create-dagster`

Dagster organise un projet en *composants*. Chaque dossier placé sous `defs/` et contenant un fichier `defs.yaml` est chargé automatiquement au démarrage. Le composant d'ingestion a été généré par la commande `dg scaffold defs dagster_dlt.DltLoadCollectionComponent mssql_ingestion`,

avec SQL Server comme source et Snowflake comme destination (figure 3.2), puis le projet dbt a été placé dans le dossier `defs/transform`.

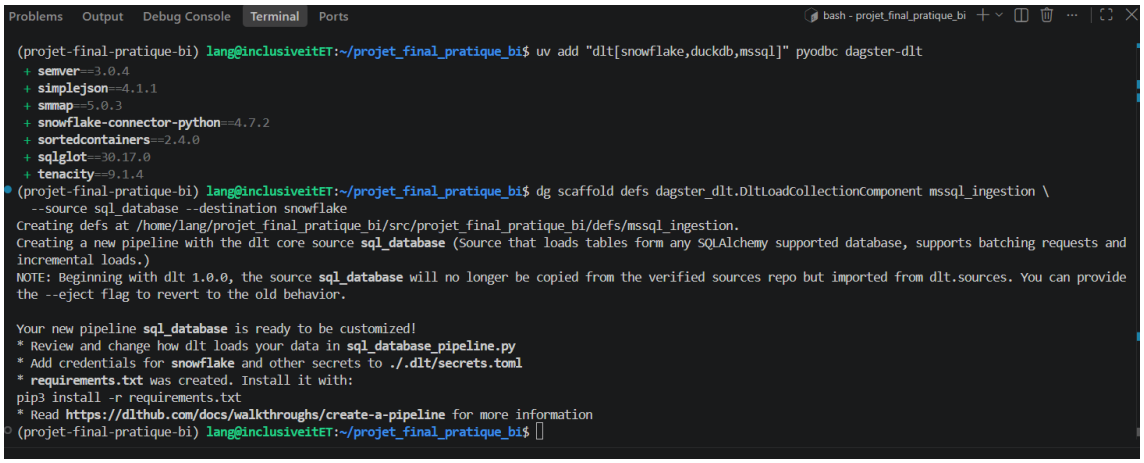


Figure 3.2 Ajout des dépendances et génération du composant DLT

La figure 3.3 montre l'arborescence obtenue. On y retrouve les deux composants (`mssql_ingestion` et `transform`), le fichier `schedules.py` qui définit la planification, et le dossier `presentation_app` qui contient l'application.

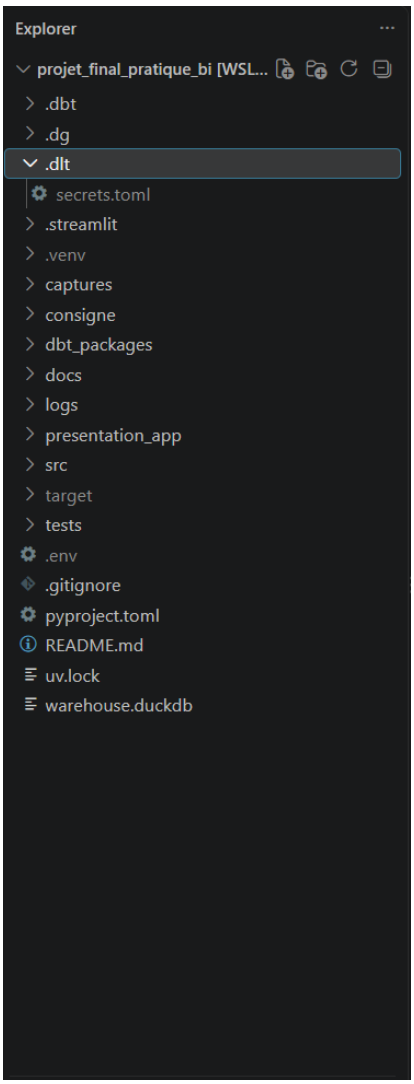


Figure 3.3 Arborescence du projet

3.2 Ingestion de la couche Bronze avec DLT

La couche Bronze reçoit les données de SQL Server sans transformation métier. Chacune des dix tables retenues au chapitre précédent est déclarée comme une ressource de la source DLT `wwi`, dans le fichier `defs/mssql_ingestion/loads.py`.

Code 3.1 Déclaration de la source DLT (defs/mssql_ingestion/loads.py)

```
@dlt.source(name="wwi")
def wwi_source():
    yield sql_table(
        credentials=credentials, schema="Sales", table="Customers",
        write_disposition="replace",
        excluded_columns=["DeliveryLocation"],
    )
    yield sql_table(
        credentials=credentials, schema="Sales", table="Orders",
        write_disposition="merge", primary_key="OrderID",
        incremental=dlt.sources.incremental("LastEditedWhen"),
```

```
    )
    # ... huit autres tables, déclarées de la même façon
```

Deux modes de chargement coexistent. Les tables de référence, peu volumineuses, sont rechargées entièrement à chaque exécution (`write_disposition="replace"`, ligne 5). Les tables de faits sont chargées de façon incrémentale : DLT mémorise la plus grande valeur de `LastEditedWhen` déjà lue et, à l'exécution suivante, ne récupère que les lignes modifiées depuis (ligne 11). Le mode `merge`, associé à la clé primaire `OrderID` (ligne 10), met à jour une commande déjà chargée au lieu de la dupliquer.

Les colonnes de type `geography` de SQL Server (`DeliveryLocation`, `Location`, `Border`) sont exclues (ligne 6), de même que les photos et les mots de passe chiffrés de la table `People`. Avant cette exclusion, chaque chargement affichait un avertissement de DLT, qui ne sait pas convertir ce type (figure 3.4).

```
(projet-final-pratique-bi) lang@inclusiveitET:~/projet_final_pratique_bi$ DLT_DESTINATION=duckdb python src/projet_final_pratique_bi/defs/mssql_ingestion/loads.py
/home/lang/projet_final_pratique_bi/.venv/lib/python3.14/site-packages/dlt/sources/sql_database/__init__.py:311: SAWarning: Did not recognize type 'geography' of column 'DeliveryLocation'
  table_obj = Table(table, metadata, autoload_with=engine, resolve_fks=resolve_foreign_keys)
2026-08-14 16:31:28,883|[WARNING]|24747|133676258559808|dlt|schema_types.py|sqla_col_to_column_schema:179|A column with name DeliveryLocation contains unknown data type NULL which cannot be mapped to `dlt` data type. When using sqlalchemy backend such data will be passed to the normalizer. In case of `pyarrow` and `pandas` backend, data types are detected from numpy ndarrays. In case of other backends, the behavior is backend-specific.
/home/lang/projet_final_pratique_bi/.venv/lib/python3.14/site-packages/dlt/sources/sql_database/__init__.py:311: SAWarning: Did not recognize type 'geography' of column 'Location'
  table_obj = Table(table, metadata, autoload_with=engine, resolve_fks=resolve_foreign_keys)
2026-08-14 16:31:58,024|[WARNING]|24747|133676258559808|dlt|schema_types.py|sqla_col_to_column_schema:179|A column with name Location contains unknown data type NULL which cannot be mapped to `dlt` data type. When using sqlalchemy backend such data will be passed to the normalizer. In case of `pyarrow` and `pandas` backend, data types are detected from numpy ndarrays. In case of other backends, the behavior is backend-specific.
```

Figure 3.4 Avertissements de DLT sur les colonnes `geography` lors des premiers chargements

Le code d'accès à SQL Server ne contient aucun mot de passe. DLT lit la chaîne de connexion dans le fichier `.dlt/secrets.toml`, exclu du dépôt Git (figure 3.5).

loads.py .gitign secrets.toml

src > projet_final_pratique_bi > defs > mssql_ingestion > .dlt > secrets.toml

```
1  [sources.sql_database.credentials]
2  drivername = "mssql+pymssql"
3  host="inclusiveit_ET"
4  port="1433"
5  database="WideWorldImporters"
6  username=[redacted]
7  password=[redacted]
8
9
10 [destination.snowflake.credentials]
11 database = "PROJET_FINAL_PRATIQUE_BI"
12 password = [redacted]
13 username = "lang"
14 host = [redacted]
15 warehouse = "COMPUTE_WH"
16 role = "PFPB_ROLE"
```

Figure 3.5 Fichier `.dlt/secrets.toml` (valeurs masquées)

La fin du fichier choisit la destination à partir de la variable `DLT_DESTINATION`.

Code 3.2 Choix de la destination et définition du pipeline (loads.py)

```
if DEST == "duckdb":
    # expandvars/expanduser : un fichier .env (chargé par Dagster) ne remplace pas $HOME ni ~.
```

```
    duckdb_path = os.path.expanduser(os.path.expandvars(os.getenv("DUCKDB_PATH",
  "warehouse.duckdb")))
    destination = dlt.destinations.duckdb(duckdb_path)
else:
    destination = "snowflake"

mssql_pipeline = dlt.pipeline(
    pipeline_name="mssql_bronze",
    destination=destination,
    dataset_name="bronze",
)
```

Le paramètre `dataset_name` (ligne 11) donne le nom du schéma cible : `bronze` dans DuckDB, `BRONZE` dans Snowflake. La ligne 3 corrige un problème rencontré pendant le développement : Dagster lit le fichier `.env` sans interpréter les variables du shell, si bien que le chemin `$HOME/...` arrivait tel quel dans le code et DuckDB créait un dossier nommé littéralement `$HOME`.

DLT normalise les identifiants : `OrderLines` devient `order_lines` et `CustomerID` devient `customer_id`. Nos premiers modèles dbt, écrits avec les noms de SQL Server, échouaient pour cette raison. Le tableau 3.1 donne les volumes chargés dans Snowflake, relevés après exécution.

Tableau 3.1 Volumes chargés dans le schéma `BRONZE` de Snowflake

Table	Lignes	Mode de chargement
`order_lines`	231 412	incrémental (`merge`)
`invoice_lines`	228 265	incrémental (`merge`)
`orders`	73 595	incrémental (`merge`)
`invoices`	70 510	incrémental (`merge`)
`cities`	37 940	complet (`replace`)
`people`	1 111	complet (`replace`)
`customers`	663	complet (`replace`)
`stock_items`	227	complet (`replace`)
`countries`	190	complet (`replace`)
`state_provinces`	53	complet (`replace`)

3.3 Transformations Silver et Gold avec dbt

3.3.1 Déclaration des sources

Le projet dbt commence par déclarer les tables Bronze comme des *sources*. Chaque source porte un nom logique, repris de SQL Server, et le nom physique créé par DLT (`identifier`).

Code 3.3 Sources Bronze déclarées dans dbt (models/_sources.yml)

```
version: 2

sources:
  - name: bronze
```

```
    schema: bronze
    tables:
      - name: Customers
        identifier: customers
      - name: Orders
        identifier: orders
      - name: OrderLines
        identifier: order_lines
```

Ce double nommage a une seconde utilité. Dagster relie un modèle dbt à l'actif DLT qui l'alimente en comparant le nom de la source à celui de la ressource DLT, qui reprend le nom de la table SQL Server. Avec `name: OrderLines` et `identifier: order_lines` (lignes 11 et 12), le lien est établi automatiquement et le lignage passe de DLT à dbt sans configuration supplémentaire.

3.3.2 Placement des modèles dans les schémas

Les modèles sont rangés par couche dans `dbt_project.yml` : le dossier `staging` produit des vues dans le schéma `silver`, le dossier `marts` des tables dans le schéma `gold`. Par défaut, dbt préfixe ce nom par le schéma de la cible et aurait créé `SILVER_SILVER` et `SILVER_GOLD`. La macro du code 3.4 remplace ce comportement.

Code 3.4 Surcharge du nommage des schémas (macros/generate_schema_name.sql)

```
{% macro generate_schema_name(custom_schema_name, node) -%}
    {%- if custom_schema_name is none -%}
        {{ target.schema }}
    {%- else -%}
        {{ custom_schema_name | trim }}
    {%- endif -%}
{%- endmacro %}
```

Quand un modèle déclare un schéma, ce nom est utilisé tel quel (ligne 5) ; sinon le modèle va dans le schéma de la cible (ligne 3).

3.3.3 Couche Silver

La couche Silver compte sept vues `stg_*`, une par table utile. Elles renomment et typent les colonnes, et effectuent les jointures purement techniques. Ainsi, la date de facturation et le client sont portés par l'en-tête de facture, alors que le bénéfice est porté par la ligne ; la vue `stg_invoice_lines` rassemble les deux.

Code 3.5 Vue Silver des lignes de facture (models/staging/stg_invoice_lines.sql)

```
with source as (
    select * from {{ source('bronze', 'InvoiceLines') }}
),
source2 as (
    select * from {{ source('bronze', 'Invoices') }}
),
```

```
renamed as (
    select
    il.invoice_line_id,
    il.invoice_id,
    i.order_id,
    i.customer_id,
    i.salesperson_person_id,
    cast(i.invoice_date as date) as invoice_date,
    il.stock_item_id,
    il.quantity,
    il.unit_price,
    il.extended_price,
    il.tax_amount,
    il.line_profit
    from source il
    join source2 i on il.invoice_id = i.invoice_id
)
select * from renamed
```

La fonction `source()` (lignes 2 et 5) désigne une table Bronze par son nom logique ; dbt en déduit l'ordre d'exécution et le lignage. La conversion de la ligne 14 ramène la date de facturation au type `DATE`, qui se joint directement à la dimension calendrier.

3.3.4 Couche Gold

La couche Gold contient les quatre dimensions et les deux tables de faits décrites au chapitre 2. Le code 3.6 montre la table `fct_sales`, qui porte la définition du chiffre d'affaires.

Code 3.6 Table de faits des ventes (models/marts/fct_sales.sql)

```
select
    ol.order_line_id,
    o.order_id,
    o.order_date,
    o.customer_id,
    o.salesperson_person_id     as salesperson_id,
    ol.stock_item_id            as product_id,
    ol.quantity,
    ol.unit_price,
    ol.quantity * ol.unit_price as sales_amount_excl_tax,
    ol.quantity * ol.unit_price * (1 + ol.tax_rate / 100) as sales_amount_incl_tax
from {{ ref('stg_order_lines') }} ol
join {{ ref('stg_orders') }} o on ol.order_id = o.order_id
```

Le grain est la ligne de commande : la jointure de la ligne 13 rattache chaque ligne à sa commande sans la dupliquer, puisqu'une ligne n'appartient qu'à une commande. Les lignes 6 et 7 renomment les clés pour qu'elles portent le même nom que dans les dimensions, ce qui simplifie les jointures écrites par le modèle de langage. Les montants hors taxes et toutes taxes comprises sont calculés une fois pour toutes (lignes 10 et 11) : aucune requête n'a à refaire ce calcul. La fonction `ref()` désigne un autre modèle dbt, ici deux vues Silver.

La dimension `dim_date` est générée par la macro `dbt.date_spine`, qui produit un jour par ligne du 1er janvier 2013 au 31 décembre 2016, soit 1 461 lignes, avec l'année, le trimestre, le mois et le jour.

3.3.5 Tests de qualité

Les tests sont déclarés dans les fichiers YAML qui documentent les modèles. Le code 3.7 en montre une partie pour la couche Gold.

Code 3.7 Documentation et tests de fct_sales (models/marts/_marts.yml)

```
models:
  - name: fct_sales
    description: "Ventes, une ligne par ligne de commande"
    columns:
      - name: order_line_id
        data_tests: [unique, not_null]
      - name: sales_amount_excl_tax
        description: "Chiffre d'affaires HT = quantity * unit_price"
      - name: product_id
        data_tests:
          - relationships:
              arguments: {to: ref('dim_products'), field: product_id}
      - name: customer_id
        data_tests:
          - relationships:
              arguments: {to: ref('dim_customers'), field: customer_id}
```

Le test de la ligne 6 vérifie le grain : si une jointure dupliquait des lignes de commande, `unique` échouerait. Les tests `relationships` (lignes 11 et 15) vérifient que chaque vente renvoie à un produit et à un client existants ; une vente orpheline disparaîtrait sinon silencieusement des analyses qui joignent les dimensions. Le projet compte dix-huit tests au total, répartis entre les couches Silver et Gold. La commande `dbt build` exécute les treize modèles et ces tests dans l'ordre du graphe, soit 31 nœuds ; un modèle n'est construit que si les tests des modèles dont il dépend ont réussi.

La figure 3.6 présente le graphe de dépendances généré par dbt.

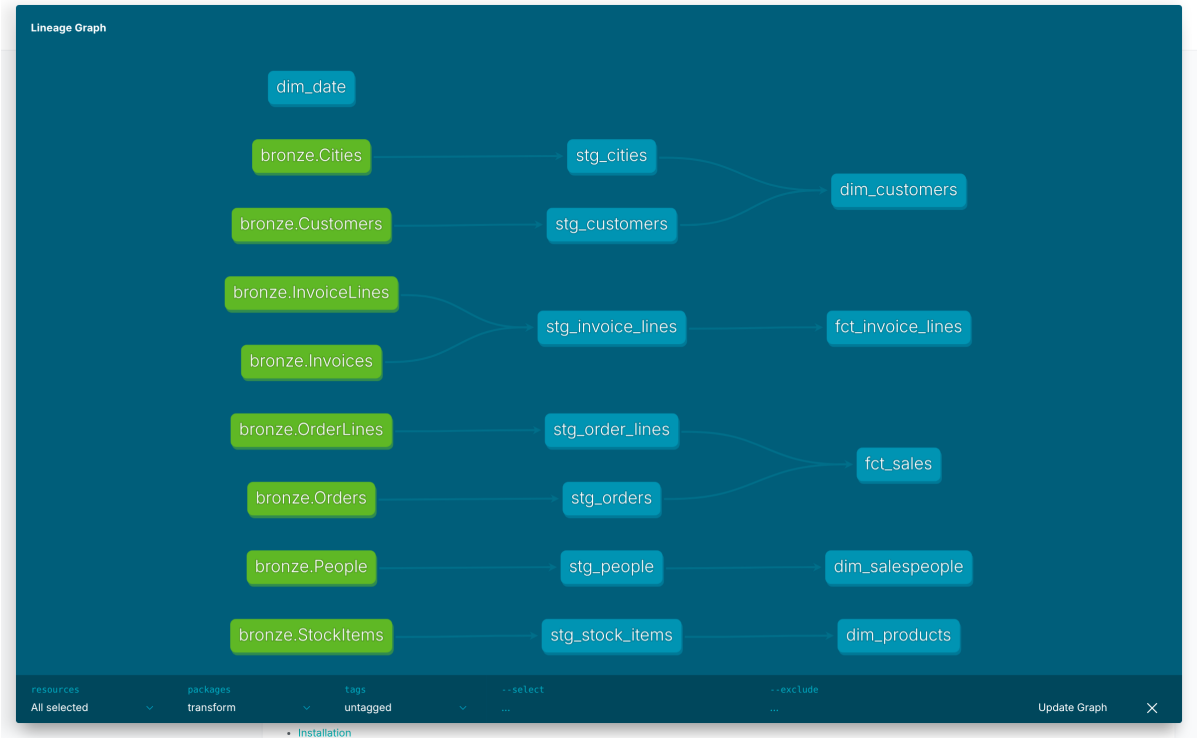


Figure 3.6 Lignage des modèles dbt, des sources Bronze aux tables Gold

3.4 Orchestration avec Dagster

3.4.1 Déclaration des composants

Les deux composants sont déclarés en YAML ; aucun code Python n'est nécessaire pour les charger.

Code 3.8 Déclaration des composants DLT et dbt (fichiers defs.yaml)

```
# defs/mssql_ingestion/defs.yaml
type: dagster_dlt.DltLoadCollectionComponent
attributes:
  loads:
    - source: .loads.mssql_source
      pipeline: .loads.mssql_pipeline

# defs/transform/defs.yaml
type: dagster_dbt.DbtProjectComponent
attributes:
  project:
    project_dir: "{{ context.project_root }}/src/projet_final_pratique_bi/defs/transform"
    profiles_dir: "{{ context.project_root }}/src/projet_final_pratique_bi/defs/transform"
  cli_args:
    - build
```

Le composant DLT pointe vers la source et le pipeline définis dans `loads.py` (lignes 5 et 6) ; chaque ressource de la source devient un actif Dagster. Le composant dbt lit le projet et

transforme chaque modèle, source et test en actif ou en contrôle (*asset check*). L'argument `build` (ligne 15) fait exécuter les tests en même temps que les modèles.

Une erreur nous a coûté du temps à cette étape : le fichier s'appelait d'abord `defs.yml`. Dagster n'accepte que l'extension `.yaml` et ignore l'autre sans message d'erreur ; l'étape dbt n'apparaissait donc pas. La commande `dg list defs` permet de vérifier ce qui est réellement chargé (figure 3.7).

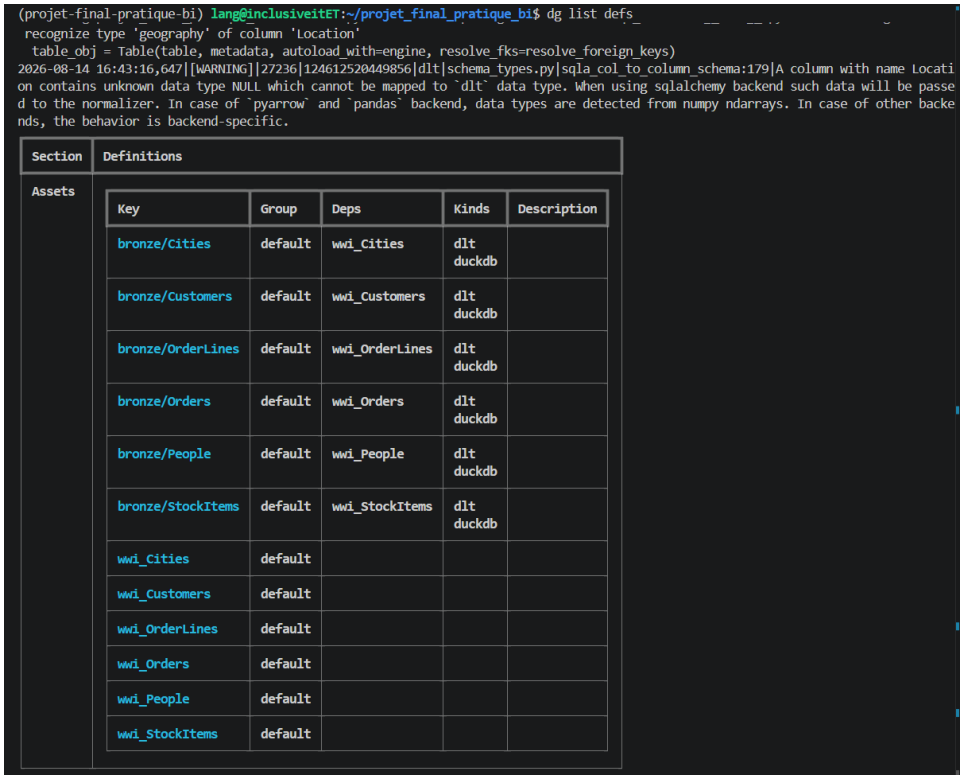


Figure 3.7 Actifs et définitions chargés, vérifiés avec `dg list defs`

3.4.2 Planification

Code 3.9 Job complet et planification quotidienne (defs/schedules.py)

```
1  import dagster as dg
2
3  pipeline_complet = dg.define_asset_job(name="pipeline_complet", selection="*")
4
5  schedule_quotidien = dg.ScheduleDefinition(
6      job=pipeline_complet,
7      cron_schedule="0 2 * * *",
8      execution_timezone="Africa/Dakar",
9  )
```

Le job `pipeline_complet` sélectionne tous les actifs (ligne 3) ; Dagster les exécute dans l'ordre du lignage, l'ingestion avant les transformations. La planification le lance chaque jour à 2 h, heure de Dakar (lignes 7 et 8), un horaire où personne n'interroge l'entrepôt.

3.4.3 Exécution

L'interface web de Dagster, lancée avec `dg dev`, affiche le graphe des actifs (figure 3.8). On y lit la chaîne complète : les dix actifs DLT de la couche Bronze, les vues Silver, puis les tables Gold.

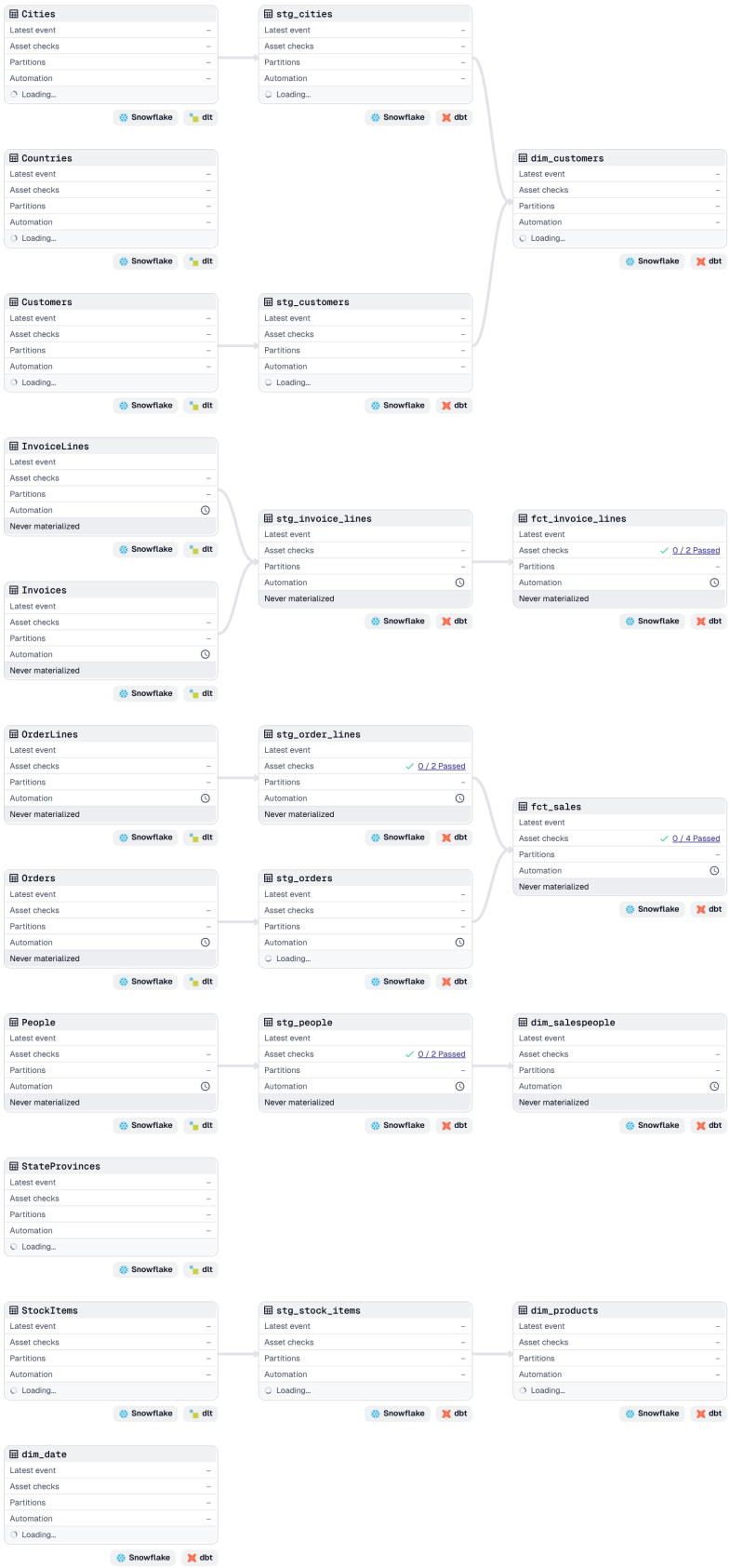


Figure 3.8 Lignage des actifs dans Dagster, de l'ingestion DLT aux tables Gold

Une exécution du job se suit étape par étape. Sur Snowflake, l'ingestion des dix tables a duré 4 min 33 s (figure 3.9) et l'étape dbt 23,28 s, contrôles compris (figure 3.10). L'exécution complète a duré 7 min 28 s (figure 3.11) ; la différence correspond au démarrage des processus et à la reprise du warehouse Snowflake.

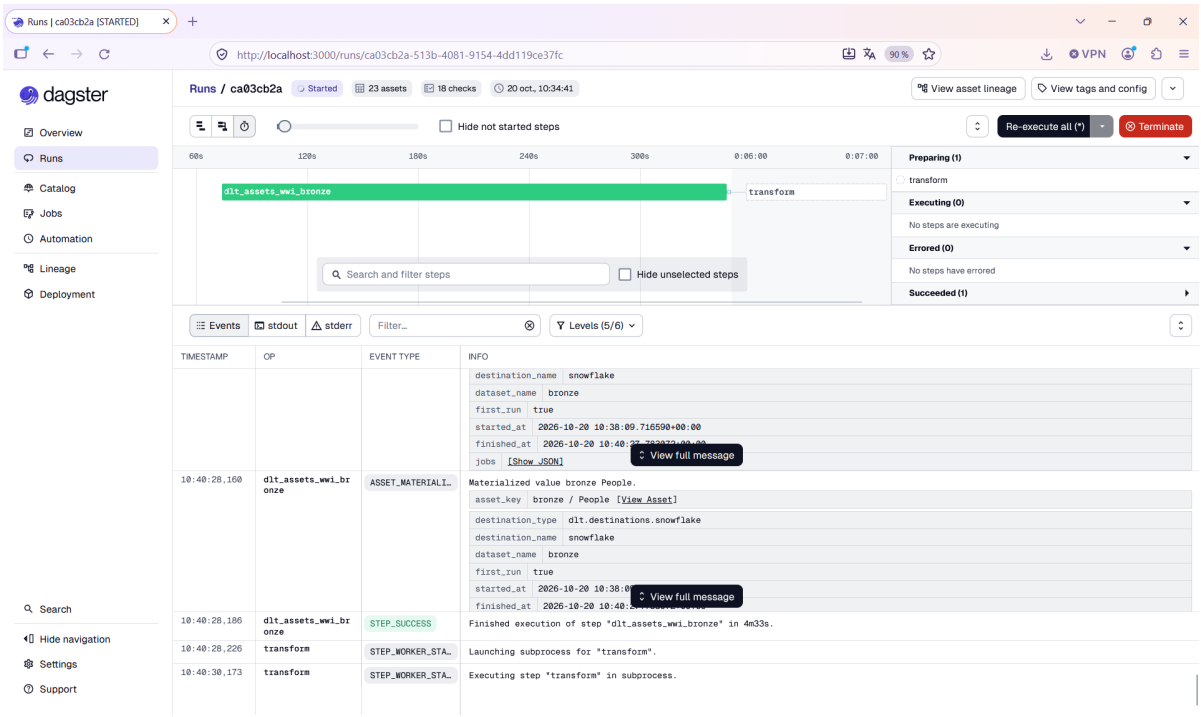


Figure 3.9 Étape d'ingestion DLT terminée

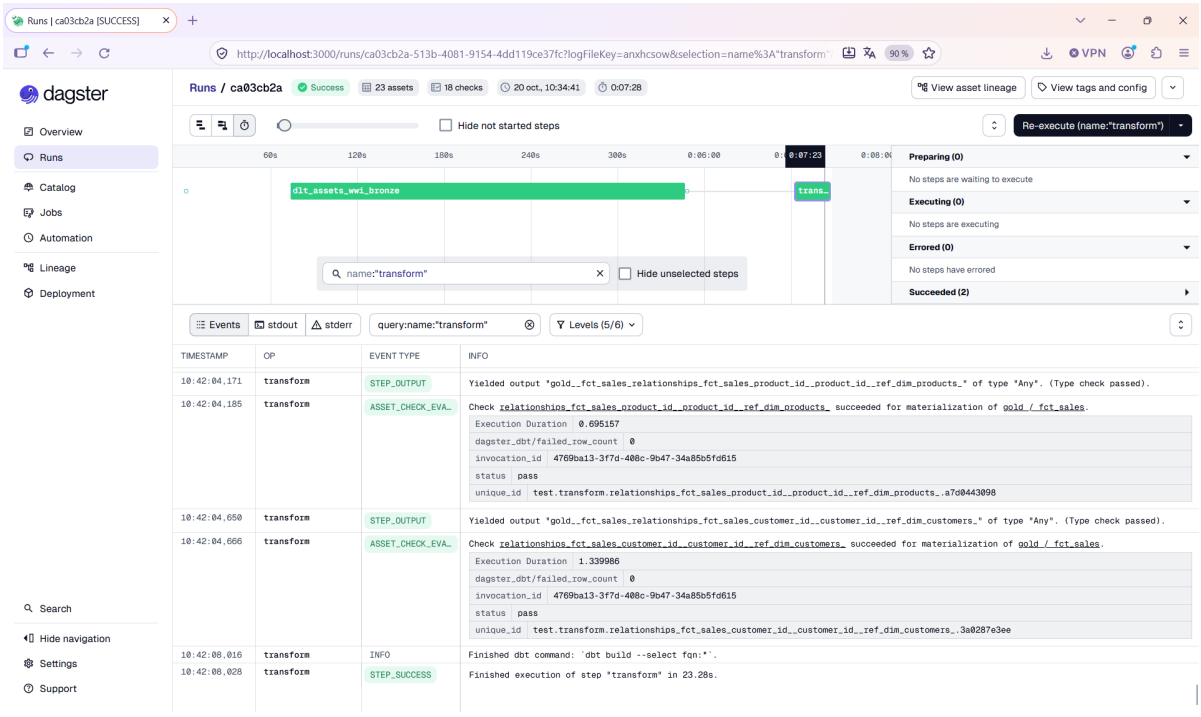


Figure 3.10 Étape dbt terminée, avec les contrôles de qualité

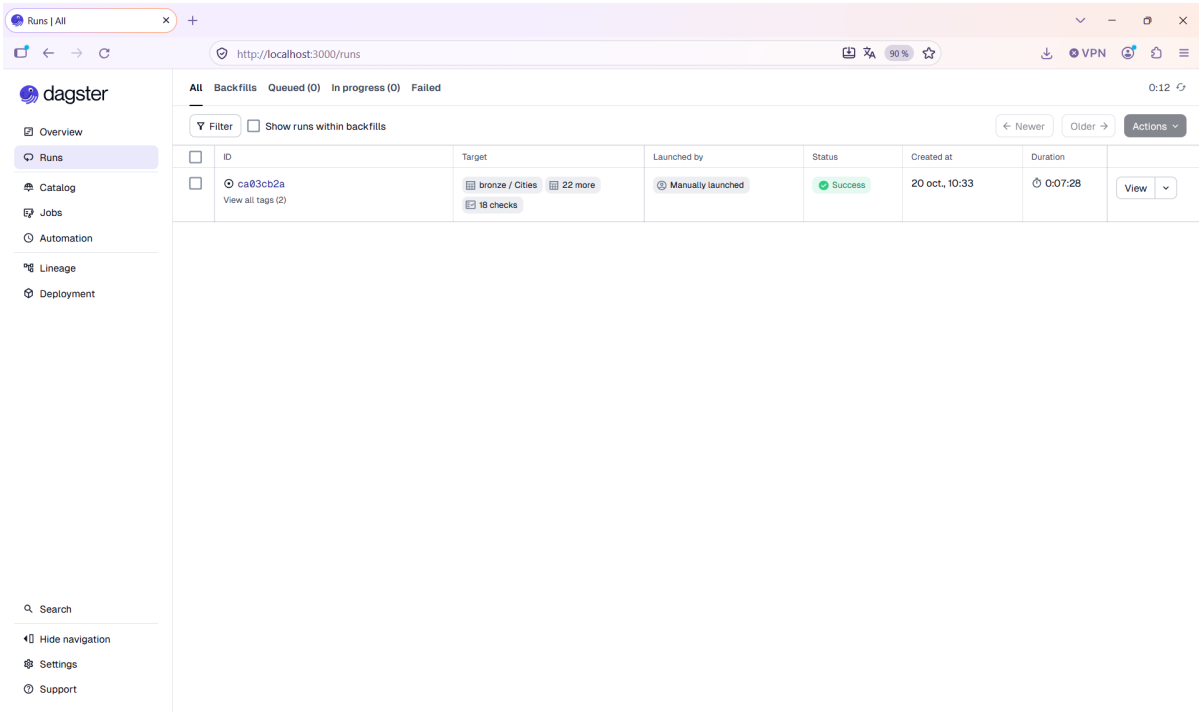


Figure 3.11 Exécution complète du job `pipeline_complet`

3.5 Passage à Snowflake

3.5.1 Droits et propriété des schémas

Le passage à Snowflake n'a demandé aucune modification du code, seulement la préparation du compte. Nous avons créé un rôle dédié, `PFPB_ROLE`, plutôt que d'utiliser `ACCOUNTADMIN`.

Code 3.10 Préparation du compte Snowflake (feuille de calcul Snowsight)

```
USE ROLE ACCOUNTADMIN;
CREATE ROLE IF NOT EXISTS PFPB_ROLE;
CREATE WAREHOUSE IF NOT EXISTS COMPUTE_WH WAREHOUSE_SIZE = XSMALL AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE;
CREATE DATABASE IF NOT EXISTS PROJET_FINAL_PRATIQUE_BI;
GRANT USAGE ON DATABASE PROJET_FINAL_PRATIQUE_BI TO ROLE PFPB_ROLE;
GRANT CREATE SCHEMA ON DATABASE PROJET_FINAL_PRATIQUE_BI TO ROLE PFPB_ROLE;
GRANT USAGE ON WAREHOUSE COMPUTE_WH TO ROLE PFPB_ROLE;
GRANT DATABASE ROLE SNOWFLAKE.CORTEX_USER TO ROLE PFPB_ROLE;
GRANT ROLE PFPB_ROLE TO USER <utilisateur_snowflake>;

USE ROLE PFPB_ROLE;
CREATE SCHEMA IF NOT EXISTS PROJET_FINAL_PRATIQUE_BI.BRONZE;
CREATE SCHEMA IF NOT EXISTS PROJET_FINAL_PRATIQUE_BI.SILVER;
CREATE SCHEMA IF NOT EXISTS PROJET_FINAL_PRATIQUE_BI.GOLD;
```

Le warehouse est le plus petit proposé (`XSMALL`) et se suspend après soixante secondes d'inactivité (ligne 3) : il ne consomme de crédits que pendant les exécutions. La ligne 8 donne au rôle l'accès aux fonctions Cortex.

L'ordre des deux blocs compte. Nous avions d'abord créé les schémas avec `ACCOUNTADMIN` : ils appartenaient alors à ce rôle, et DLT comme dbt échouaient avec l'erreur *Insufficient privileges to operate on schema*, même après plusieurs `GRANT`. Dans Snowflake, créer et remplacer des tables dans un schéma relève du propriétaire. Les schémas sont donc créés par `PFPB_ROLE` lui-même (lignes 11 à 14) ; pour des schémas existants, `GRANT OWNERSHIP ON SCHEMA ... TO ROLE PFPB_ROLE COPY CURRENT GRANTS` produit le même effet. La figure 3.12 montre le schéma `GOLD` après exécution, avec `PFPB_ROLE` comme propriétaire.

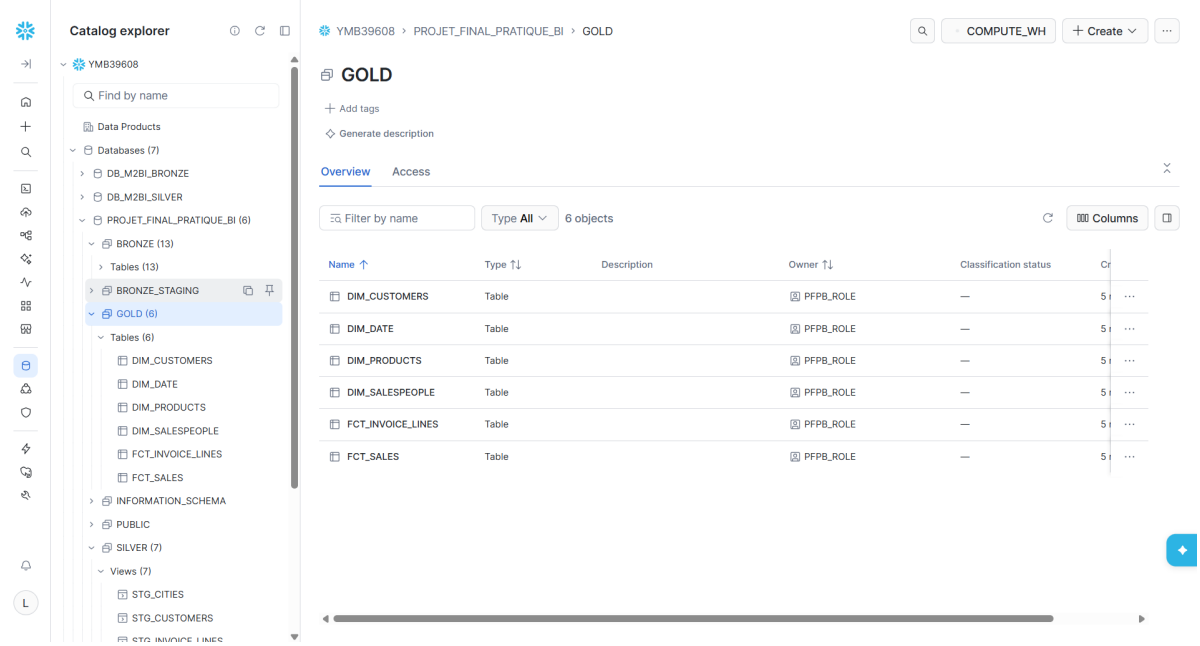


Figure 3.12 Tables du schéma `GOLD` dans Snowsight, propriété de `PFPB_ROLE`

3.5.2 Profil dbt

Le profil dbt décrit les deux cibles ; la variable `DBT_TARGET` choisit entre elles.

Code 3.11 Profil dbt à deux cibles (defs/transform/profiles.yml)

```
transform:
  target: "{{ env_var('DBT_TARGET', 'dev') }}"
  outputs:
    dev:
      type: duckdb
      path: "{{ env_var('DUCKDB_PATH') | replace('$HOME', env_var('HOME')) }}"
      schema: main
      threads: 4
    prod:
      type: snowflake
      account: "{{ env_var('SF_ACCOUNT') }}"
      user: "{{ env_var('SF_USER') }}"
      password: "{{ env_var('SF_PASSWORD') }}"
      role: PFPB_ROLE
      warehouse: "{{ env_var('SF_WAREHOUSE', 'COMPUTE_WH') }}"
      database: PROJET_FINAL_PRATIQUE_BI
      schema: SILVER
```

```
18        threads: 4
```

Les identifiants sont lus dans l'environnement (lignes 11 à 13) et n'apparaissent jamais dans le dépôt. Le filtre `replace` de la ligne 6 corrige, côté dbt, le même problème de `$HOME` que dans `loads.py`. La commande `dbt debug --target prod` a confirmé la connexion avant la première exécution (figure 3.13).

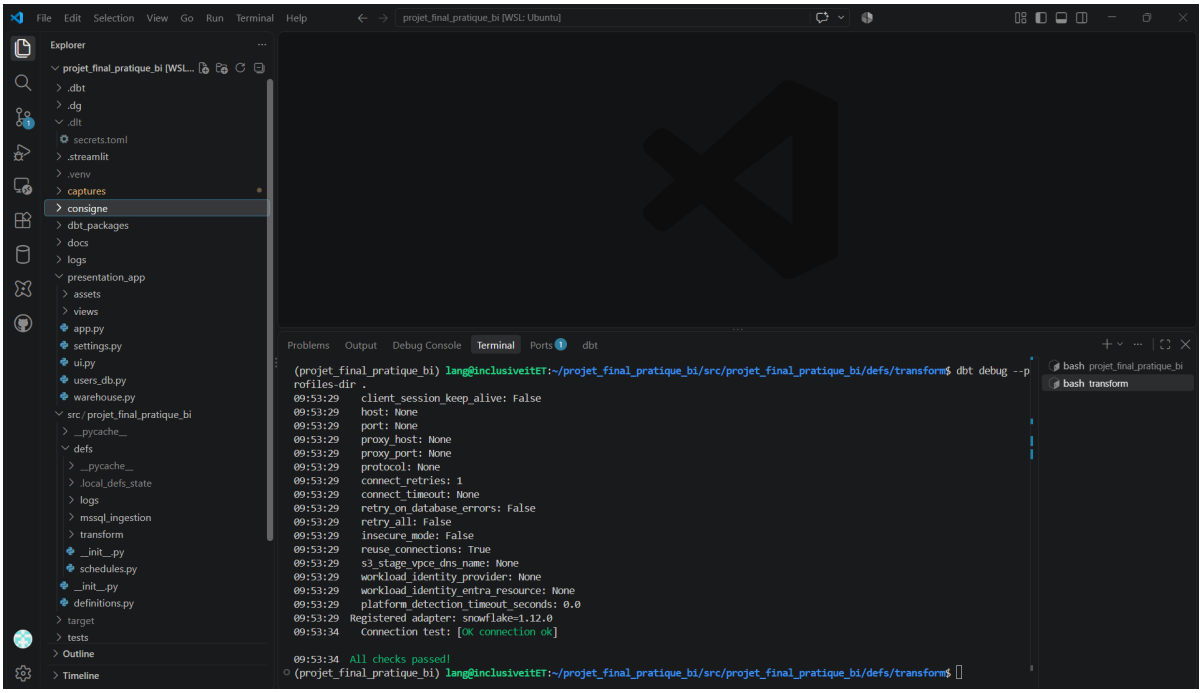


Figure 3.13 Vérification de la connexion à Snowflake avec `dbt debug`

Après l'exécution, une requête sur les vues `INFORMATION_SCHEMA` a permis de comparer les volumes des trois couches (figure 3.14). La figure 3.15 montre un aperçu de `FCT_SALES` dans Snowsight ; la description de la table a été proposée par la fonction de documentation automatique de Snowflake.

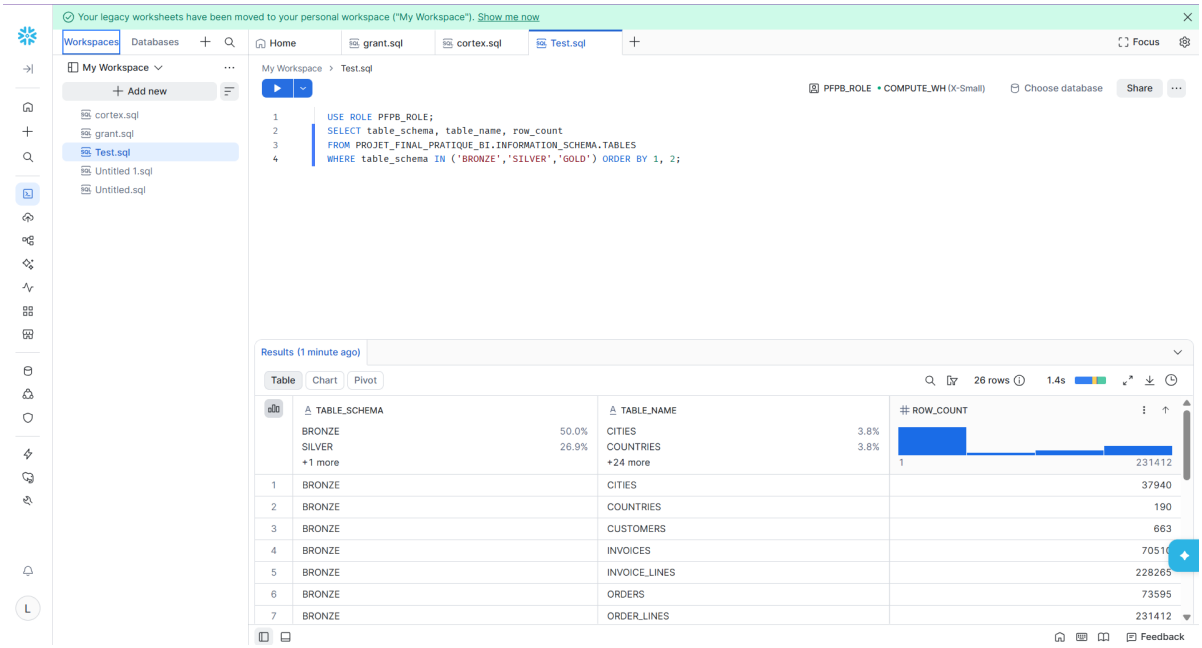


Figure 3.14 Nombre de lignes par table dans les trois schémas

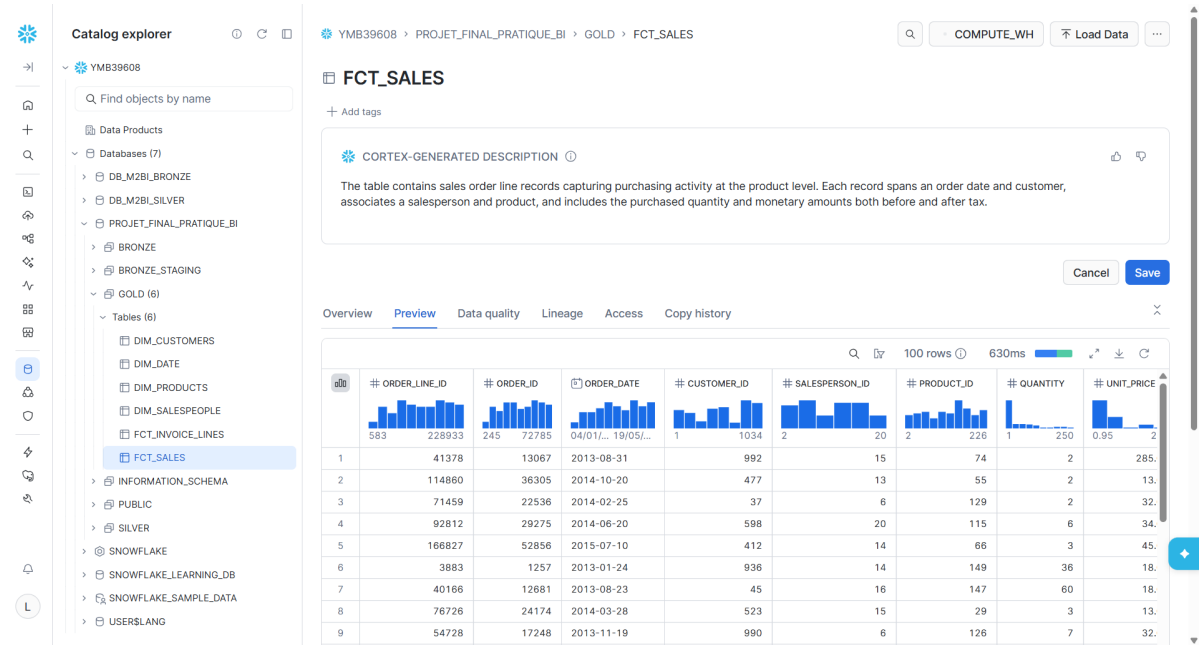


Figure 3.15 Aperçu de la table `FCT_SALES` dans Snowsight

3.6 Application Text-to-SQL WWI Insight

3.6.1 Organisation du code

L'application est écrite avec Streamlit et répartie en modules courts :

- `settings.py` lit la configuration dans les variables d'environnement ;
- `warehouse.py` génère, contrôle et exécute le SQL ;
- `users_db.py` gère les comptes, les sessions, les dossiers et les analyses dans une base

SQLite, séparée de l'entrepôt ;

- `session.py` gère la navigation et la session persistante ;
- `views/` contient une page par fichier : accueil, connexion, espace d'analyse, à propos.

Elle fonctionne sur Snowflake comme sur DuckDB : si les identifiants Snowflake sont présents, elle se connecte au schéma `GOLD` avec le rôle `PFPB_ROLE` ; sinon, elle ouvre le fichier DuckDB local en lecture seule. La page d'accueil affiche quelques chiffres clés calculés directement sur la couche Gold (figure 3.16).

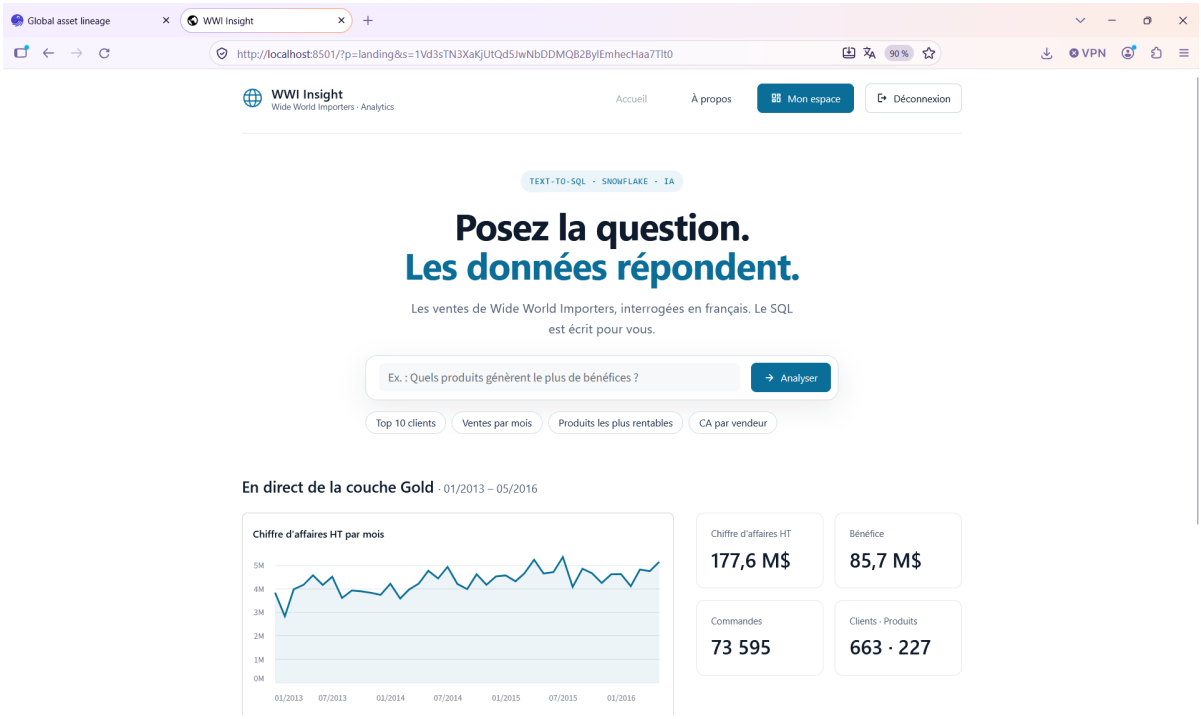


Figure 3.16 Page d'accueil connectée à Snowflake

3.6.2 Consigne transmise au modèle

La qualité du SQL produit dépend surtout de la consigne. Celle de l'application est courte.

Code 3.12 Consigne envoyée au modèle de langage (presentation_app/warehouse.py)

```
PROMPT = """Tu es un expert SQL {dialect}. Tu réponds aux questions sur l'entreprise
Wide World Importers à partir de ce schéma (couche GOLD) :

{schema}

Règles :
- une seule requête SELECT, uniquement sur les tables gold ci-dessus ;
- chiffre d'affaires = sum(sales_amount_excl_tax) de gold.fct_sales ;
- bénéfice = sum(line_profit) de gold.fct_invoice_lines ;
- noms de colonnes de résultat en français et en snake_case ;
- {max_rows} lignes maximum.
Réponds uniquement avec le SQL, sans explication ni balise.

Question : {question}"""
```

Le dialecte (ligne 1) vaut `Snowflake` ou `DuckDB`, les deux n'ayant pas exactement les mêmes fonctions de date. Le schéma (ligne 4) n'est pas écrit à la main : il est lu dans `information_schema.columns` au démarrage, puis réduit à une ligne par table avec des types simplifiés (`texte`, `nombre`, `date`). Une consigne plus courte est traitée plus vite par un modèle local. Les lignes 8 et 9 fixent les deux définitions métier. Sans elles, le modèle doit deviner quelle table de faits utiliser : il pourrait calculer le chiffre d'affaires à partir des factures, ou chercher un bénéfice dans `fct_sales`, qui n'en contient pas.

3.6.3 Choix du moteur de génération

Le compte d'essai Snowflake refuse l'appel à `SNOWFLAKE.CORTEX.COMPLETE`. Plutôt que de lier l'application à un seul moteur, nous la faisons choisir au démarrage.

Code 3.13 Sélection du moteur de génération (warehouse.py)

```
@st.cache_data(ttl=600, show_spinner=False)
def generator() -> str | None:
    """Moteur de génération utilisé : "cortex", "ollama" ou None (exemples seulement)."""
    if SQL_GENERATOR in ("cortex", "ollama"):
        return SQL_GENERATOR
    if SQL_GENERATOR == "none":
        return None
    if _cortex_available():
        return "cortex"
    if _ollama_available():
        # Préchargement du modèle en arrière-plan : la première question n'attend pas son chargement.
        threading.Thread(target=_warm_up, daemon=True).start()
        return "ollama"
    return None
```

La variable `SQL_GENERATOR` permet d'imposer un moteur (lignes 4 à 7). Par défaut, l'application tente un appel minimal à Cortex (ligne 8) ; en cas d'échec, elle vérifie qu'un serveur Ollama tourne sur le poste et qu'il dispose du modèle `qwen2.5-coder:7b` (ligne 10). Sans aucun moteur, seules les questions d'exemple restent disponibles, avec leur SQL écrit à l'avance. Le résultat est gardé en cache dix minutes (ligne 1), pour ne pas refaire ces vérifications à chaque question. La ligne 12 lance une première requête vide en arrière-plan : Ollama charge alors le modèle en mémoire avant que l'utilisateur pose sa première question.

Le SQL généré est lui aussi mis en cache, par question. Une question déjà posée obtient sa réponse en environ 0,3 s, contre une trentaine de secondes pour une génération par Ollama sur le processeur de notre machine.

3.6.4 Contrôle des requêtes

Une requête produite par un modèle de langage ne peut pas être exécutée telle quelle : le modèle peut ajouter du texte autour du SQL, produire plusieurs instructions ou, sur une ques-

tion mal formulée, proposer une modification des données. Chaque requête passe donc par la fonction `clean_sql`.

Code 3.14 Contrôle d'une requête avant exécution (warehouse.py)

```
FORBIDDEN = re.compile(
    r"\b(insert|update|delete|merge|drop|alter|create|truncate|grant|revoke|"
    r"copy|put|call|execute|attach|detach|install|pragma)\b",
    re.IGNORECASE,
)

def clean_sql(sql: str) -> str:
    sql = re.sub(r"```(?:sql)?", "", sql, flags=re.IGNORECASE).strip().rstrip(";").strip()
    if not re.match(r"^(select|with)\b", sql, re.IGNORECASE):
        raise QueryError("Requête refusée : seules les requêtes SELECT sont autorisées.")
    if ";" in sql:
        raise QueryError("Requête refusée : une seule instruction est autorisée.")
    if FORBIDDEN.search(re.sub(r"'[^']*'", "''", sql)):
        raise QueryError("Requête refusée : elle contient une instruction d'écriture.")
    if not re.search(r"\blimit\s+\d+\s*$", sql, re.IGNORECASE):
        sql = f"{sql}\nlimit {MAX_ROWS}"
    return sql
```

La ligne 8 retire les balises Markdown et le point-virgule final. La requête doit ensuite commencer par `SELECT` ou `WITH` (ligne 9), et une seconde instruction est rejetée (ligne 11). Les mots-clés d'écriture sont recherchés après avoir vidé les chaînes de caractères (ligne 13) : une condition comme `WHERE description = 'drop shipping'` reste acceptée. Enfin, une clause `LIMIT` est ajoutée si elle manque (lignes 15 et 16), ce qui borne le résultat à mille lignes. La figure 3.17 montre le refus d'une requête d'écriture.

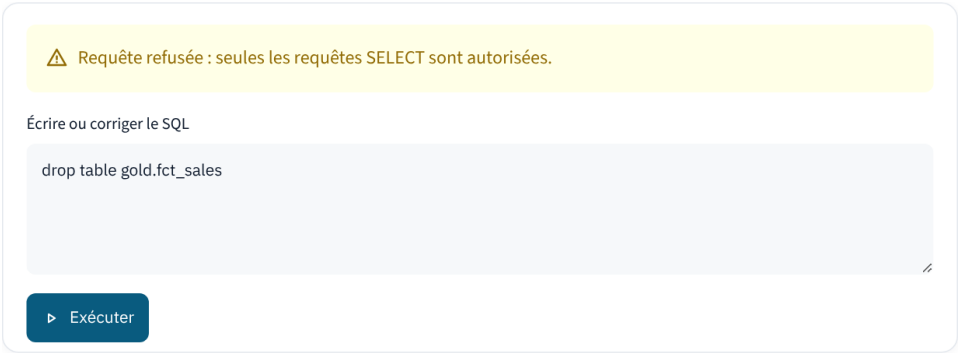


Figure 3.17 Requête d'écriture refusée par l'application

Ce filtre ne remplace pas les droits de la base. En production, l'application devrait se connecter avec un rôle limité à la lecture du schéma `GOLD` ; nous y revenons dans les limites du chapitre 4.

3.6.5 Sessions et dossiers

Streamlit perd son état à chaque rechargement de la page. Pour qu'un utilisateur ne soit pas déconnecté en actualisant son navigateur, l'application crée à la connexion un jeton aléatoire,

enregistré dans SQLite avec une date d'expiration, et le place dans l'adresse de la page.

Code 3.15 Création et restauration d'une session (users_db.py et session.py)

```
SESSION_DAYS = 7

def create_session(user_id: int) -> str:
    token = secrets.token_urlsafe(32)
    expires = (datetime.now(timezone.utc) +
  timedelta(days=SESSION_DAYS)).isoformat(timespec="seconds")
    with _connect() as con:
        con.execute("delete from sessions where expires_at < ?", (_now(),))
        con.execute("insert into sessions (token, user_id, expires_at) values (?, ?, ?)",
                    (token, user_id, expires))
    return token

def restore() -> None:
    if "user" not in st.session_state and (token := st.query_params.get("s")):
        user = users_db.session_user(token)
        if user:
            st.session_state.user, st.session_state.token = user, token
        else:
            del st.query_params["s"]
```

Le jeton fait 32 octets aléatoires (ligne 4) et expire au bout de sept jours (lignes 1 et 5) ; les sessions expirées sont purgées à chaque nouvelle connexion (ligne 7). Au chargement de la page, `restore` relit le jeton dans le paramètre `s` de l'adresse et retrouve l'utilisateur (lignes 13 et 14). Un jeton inconnu ou expiré est retiré de l'adresse (ligne 18). La déconnexion supprime la ligne de la table `sessions`. Les mots de passe sont stockés sous forme de hachés `bcrypt`.

Chaque utilisateur range ses analyses dans des dossiers. Un dossier vide propose les questions d'exemple, regroupées par thème (figure 3.18). Chaque analyse enregistrée conserve la question, le SQL exécuté, le résultat, le moteur utilisé et la durée de traitement.

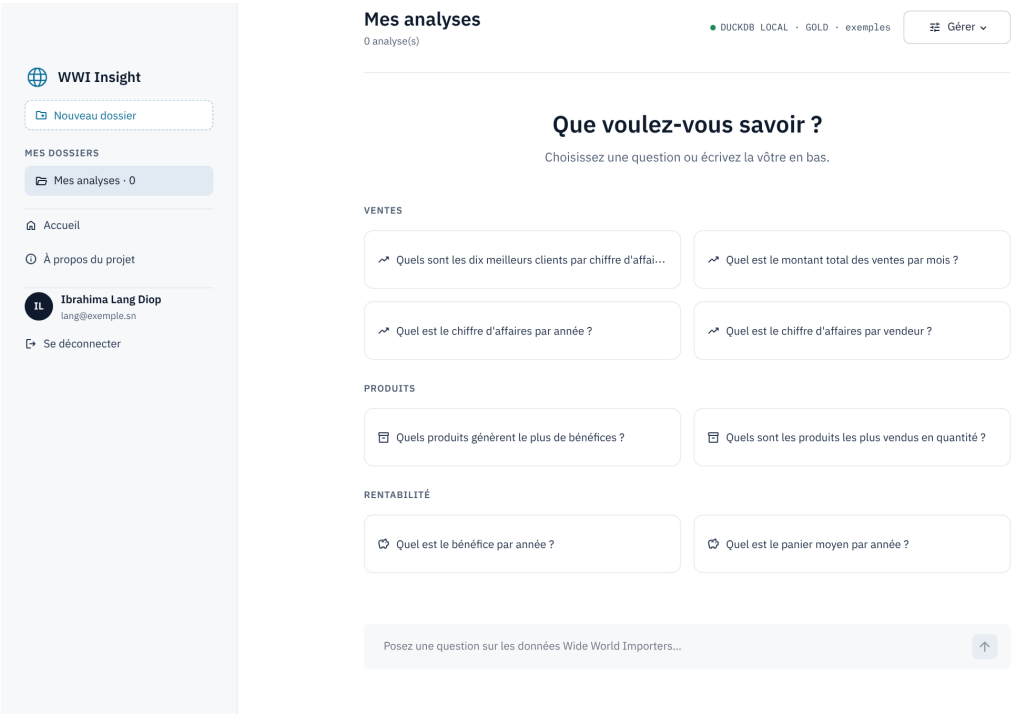


Figure 3.18 Dossier vide et questions d'exemple

3.6.6 Une question traitée de bout en bout

La figure 3.19 montre une question libre posée sur Snowflake, sans correspondance avec les exemples : « Les 10 produits les plus vendus ? ». Le SQL a été généré par Ollama en 36,5 s. L'application choisit le type de graphique d'après les colonnes du résultat, ici un diagramme en barres horizontales, et affiche une courte phrase de synthèse. L'onglet SQL (figure 3.20) permet de vérifier la requête : elle somme la colonne `quantity` de `fct_sales`, joint `dim_products` sur la bonne clé et trie par quantité décroissante. Le premier produit, *Black and orange fragile despatch tape 48mmx75m*, avec 207 324 unités, correspond au résultat d'une requête écrite à la main.

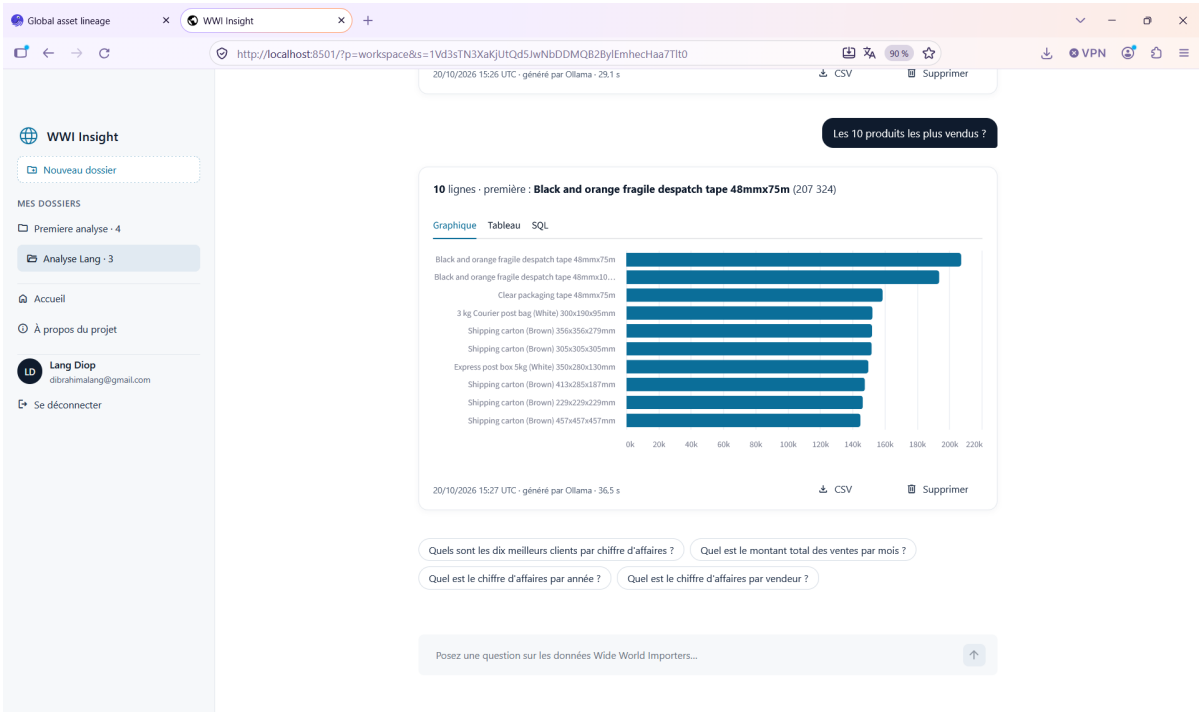


Figure 3.19 Réponse à une question libre, SQL généré par Ollama

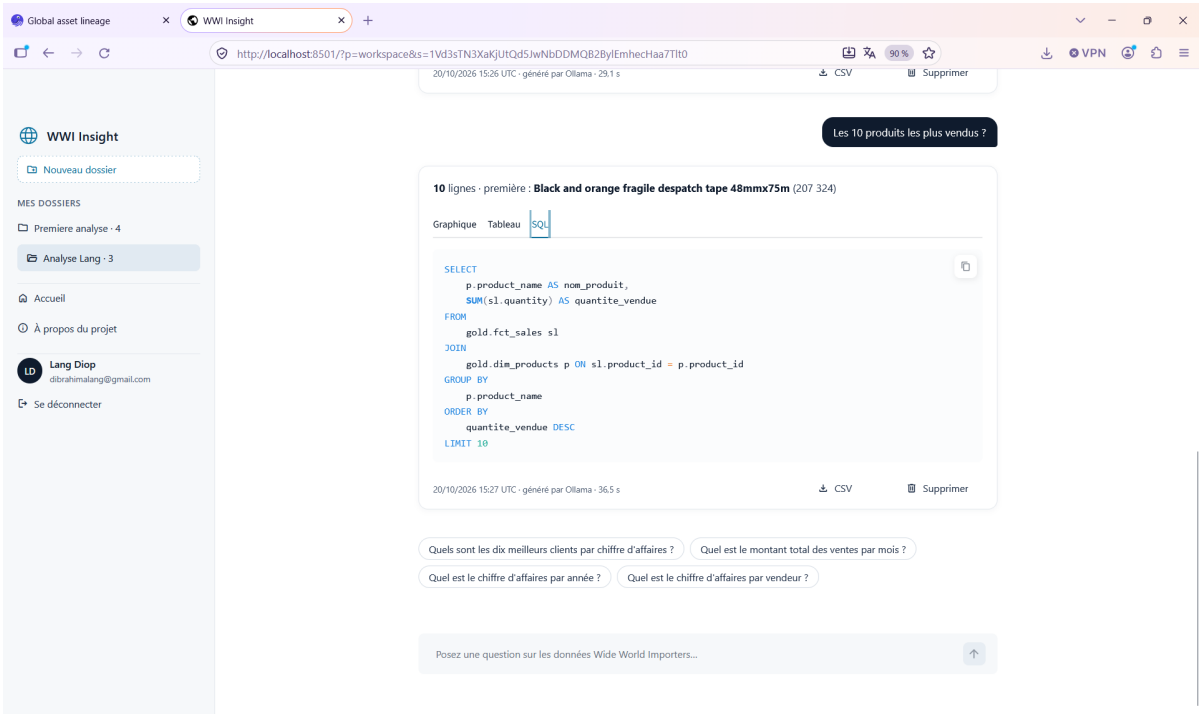


Figure 3.20 Requête générée, affichée dans l'onglet SQL

La page « À propos » (figure 3.21) rappelle le cadre du projet et l'architecture.

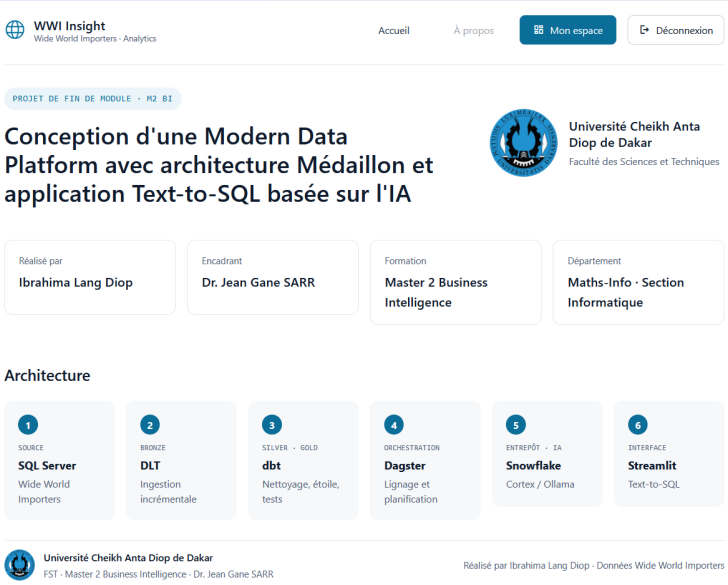


Figure 3.21 Page « À propos »

CHAPITRE 4

Résultats et discussion

4.1 Volumes et qualité des données

La chaîne complète s'exécute sans erreur sur Snowflake. Le tableau 4.1 donne le contenu de la couche Gold après exécution.

Tableau 4.1 Contenu de la couche Gold dans Snowflake

Table	Lignes	Contenu
`fct_sales`	231 412	une ligne par ligne de commande
`fct_invoice_lines`	228 265	une ligne par ligne de facture
`dim_date`	1 461	un jour par ligne, de 2013 à 2016
`dim_customers`	663	clients et ville de livraison
`dim_products`	227	articles en stock
`dim_salespeople`	10	personnes ayant le statut de vendeur

Les volumes des tables de faits sont identiques à ceux des tables Bronze correspondantes : 231 412 lignes de commande et 228 265 lignes de facture. Les jointures de la couche Silver n'ont donc ni perdu ni dupliqué de lignes, ce que confirment aussi les tests `unique` sur les clés des faits. Les dix-huit tests de qualité réussissent tous lors de l'exécution de référence.

4.2 Performances

Le tableau 4.2 regroupe les durées mesurées. Celles de la chaîne proviennent de Dagster ; celles de l'application sont enregistrées avec chaque analyse.

Tableau 4.2 Durées mesurées sur la plateforme

Opération	Durée
Ingestion DLT des dix tables (chargement initial)	4 min 33 s
Construction et tests dbt (31 nœuds)	23,28 s
Exécution complète du job `pipeline_complet`	7 min 28 s
Génération du SQL par Ollama (questions libres)	29 à 37 s
Question déjà posée (SQL en cache)	environ 0,3 s

L'ingestion représente l'essentiel du temps. Les exécutions suivantes seront plus courtes, puisque les quatre tables de faits ne transfèrent alors que les lignes modifiées. Les transformations dbt sont rapides, même sur le plus petit warehouse Snowflake.

La latence de l'application est dominée par la génération du SQL. Ollama tourne ici sur le processeur d'un ordinateur portable, sans carte graphique : un modèle de sept milliards de paramètres y produit une requête en une trentaine de secondes.

4.3 Réponses aux questions de la consigne

Les trois questions de la consigne ont été posées dans l'application. Les figures de cette section proviennent d'une exécution sur la base DuckDB locale, qui contient les mêmes données que Snowflake ; les montants ont été vérifiés sur les deux.

4.3.1 Les dix meilleurs clients

Le meilleur client est *Tailspin Toys (Inguadona, MN)*, avec 384 393 $ de chiffre d'affaires hors taxes sur la période (figure 4.1). L'écart entre les dix premiers clients est faible, et ils ne représentent ensemble que 3,71 M$, soit 2,1 % du chiffre d'affaires total de 177,6 M$. La clientèle de Wide World Importers est donc très dispersée : aucun client ne pèse assez pour mettre l'entreprise en difficulté s'il partait.

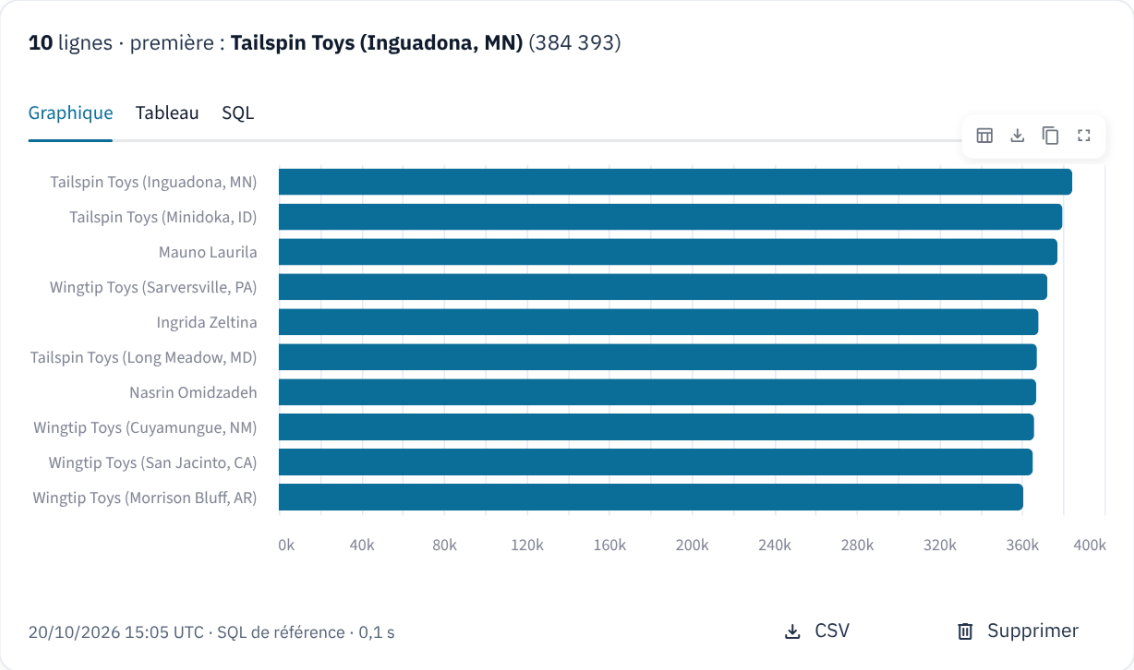


Figure 4.1 Les dix meilleurs clients par chiffre d'affaires (exécution locale DuckDB)

Le tableau de résultat et le SQL exécuté sont accessibles dans deux autres onglets (figure 4.2).



Figure 4.2 Requête exécutée pour la première question

4.3.2 Les ventes par mois

Les ventes couvrent 41 mois, de janvier 2013 à mai 2016 (figure 4.3). Le chiffre d'affaires mensuel moyen est de 4,33 M$. Le mois le plus faible est février 2013 (2,82 M$) et le plus fort juillet 2015 (5,34 M$). La tendance est à la hausse : le chiffre d'affaires annuel passe de 46,9 M$ en 2013 à 51,5 M$ en 2014 puis 55,8 M$ en 2015, soit environ 9 % de croissance par an. L'année 2016 n'est couverte que sur cinq mois (23,4 M$).

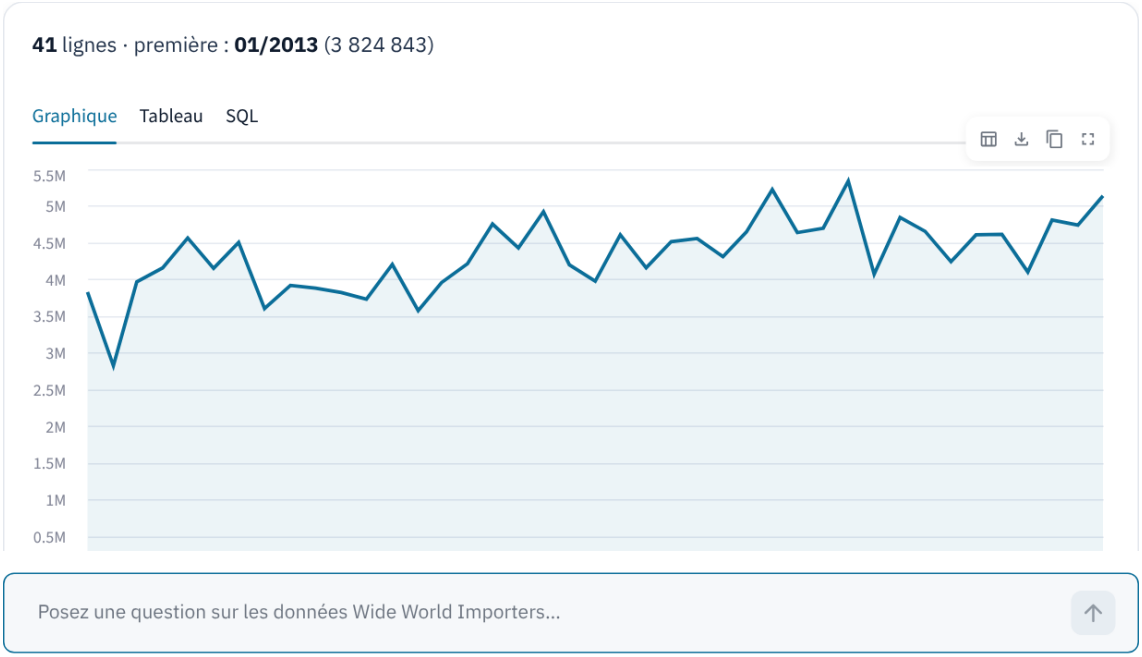


Figure 4.3 Chiffre d'affaires par mois (exécution locale DuckDB)

4.3.3 Les produits les plus rentables

Le bénéfice total s'élève à 85,7 M$, soit environ 48 % du chiffre d'affaires hors taxes. Les produits les plus rentables sont des articles d'emballage : le papier bulle double face de 20 mm en rouleau de 50 m (5,29 M$ de bénéfice) et la machine à coussins d'air bleue (4,44 M$), comme le montre la figure 4.4. Cette réponse n'était possible qu'avec la seconde table de faits, puisque le bénéfice n'est connu qu'à la facturation.

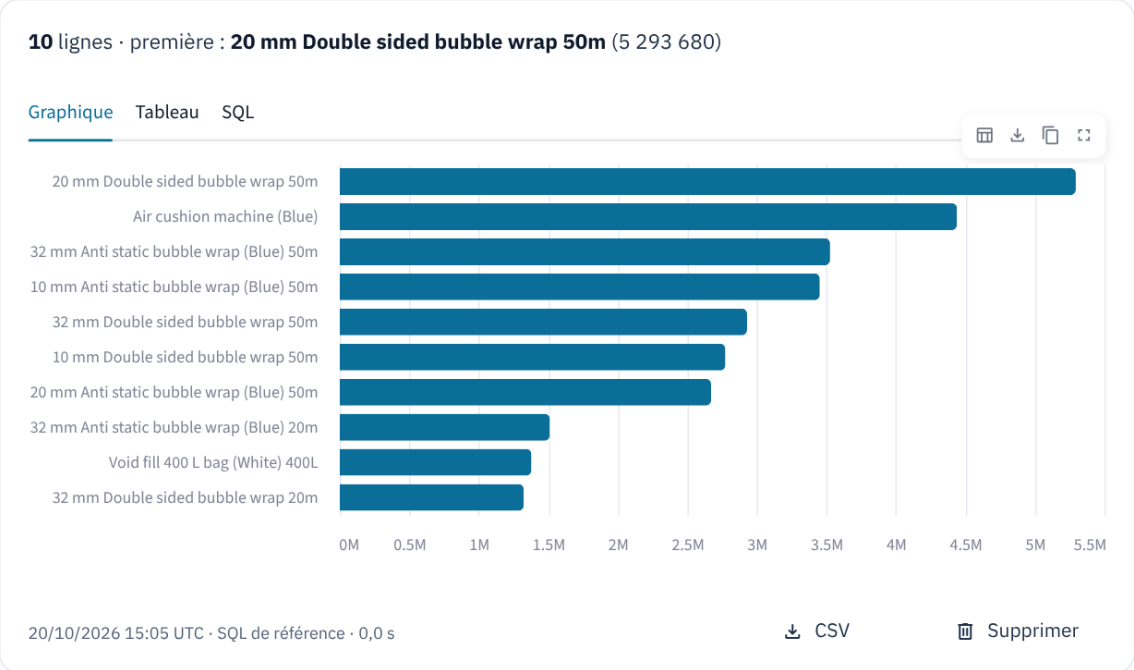


Figure 4.4 Les produits qui génèrent le plus de bénéfice (exécution locale DuckDB)

4.4 Évaluation du Text-to-SQL

Les huit questions d'exemple ont un SQL écrit et vérifié à la main ; elles ne mesurent donc pas le modèle. Pour les questions libres, nous avons testé un petit nombre de formulations sur Snowflake avec Ollama, par exemple « Les 10 produits les plus vendus ? », présentée au chapitre 3. Sur ces essais, le SQL produit était correct : bonnes tables, bonnes jointures et bonne mesure, avec un résultat identique à celui d'une requête écrite à la main.

Ce constat doit rester prudent. Le nombre de questions testées est trop faible pour donner un taux de réussite, et elles portaient toutes sur des agrégations simples. Une évaluation sérieuse demanderait un jeu d'une cinquantaine de questions avec leur réponse attendue, comparées automatiquement au résultat de l'application. Elle permettrait aussi de mesurer l'effet des règles métier de la consigne (code 3.12), que nous avons ajoutées par précaution sans en mesurer l'apport.

4.5 Difficultés rencontrées

Le tableau 4.3 reprend les principaux problèmes rencontrés et la solution retenue pour chacun.

Tableau 4.3 Difficultés rencontrées et solutions retenues

Problème	Solution
Modèles dbt en échec : noms de tables introuvables	Sources dbt avec `identifier` (noms normalisés par DLT)
Étape dbt absente de Dagster	Fichier renommé de `defs.yml` en `defs.yaml`
Dossier nommé `$HOME` créé par DuckDB	`expandvars` dans le code, `replace` dans le profil dbt
Insufficient privileges sur les schémas Snowflake	Schémas créés par `PFPB_ROLE`, ou propriété transférée
Avertissements DLT sur les colonnes `geography`	Colonnes exclues de l'ingestion
Cortex `COMPLETE` refusé sur le compte d'essai	Repli automatique sur Ollama
Déconnexion à chaque rechargement de la page	Jeton de session dans l'adresse, stocké dans SQLite
Génération lente lors de la première question	Préchargement du modèle, cache du SQL généré

4.6 Limites

La solution présente plusieurs limites, dont certaines tiennent aux conditions du projet.

- **Cortex n'a pas pu être utilisé.** Le compte d'essai Snowflake ne donne pas accès à `CORTEX.COMPLETE`. Le code d'appel existe et le rôle dispose du droit `CORTEX_USER`, mais il n'a pas été testé sur un compte payant.
- **La génération locale est lente.** Une trentaine de secondes par question reste acceptable pour une démonstration, pas pour un usage quotidien. Une carte graphique ou un modèle hébergé réduirait ce délai à quelques secondes.
- **L'application utilise un rôle trop large.** Elle se connecte avec `PFPB_ROLE`, qui peut écrire dans les trois schémas. Le filtre `clean_sql` bloque les requêtes d'écriture, mais une protection applicative ne vaut pas un droit refusé par la base. Un rôle dédié, limité à `SELECT` sur `GOLD`, serait nécessaire en production.
- **Le jeton de session apparaît dans l'adresse.** Il peut se retrouver dans l'historique du navigateur ou être partagé par erreur avec un lien. Un cookie sécurisé serait préférable, mais Streamlit ne permet pas d'en poser simplement.
- **L'évaluation du Text-to-SQL est réduite**, comme indiqué plus haut.
- **Le modèle Gold ne garde pas l'historique des dimensions.** Si un client change de ville, l'ancienne valeur est écrasée. Une dimension à variation lente (*snapshot* dbt) corrigerait ce point.

Conclusion générale

Ce projet avait pour but de construire une plateforme de données complète, de la base transactionnelle jusqu'à une application interrogeable en français. La chaîne fonctionne de bout en bout sur Snowflake : DLT charge dix tables de *WideWorldImporters* dans la couche Bronze, dbt construit sept vues Silver et six tables Gold vérifiées par dix-huit tests, et Dagster enchaîne ces étapes en un peu plus de sept minutes, chaque nuit à 2 h. L'application WWI Insight répond aux trois questions de la consigne et à des questions libres, en contrôlant chaque requête avant de l'exécuter.

Trois choix ont particulièrement compté. Le développement sur DuckDB a permis de mettre au point les modèles et l'application sans dépenser de crédits, puis de passer à Snowflake en changeant seulement des variables d'environnement. Le modèle en étoile à deux tables de faits a résolu la question du bénéfice, qui n'existe qu'à la facturation, et il a donné au modèle de langage un schéma simple à interroger. Enfin, la sélection automatique du moteur de génération a permis de contourner l'absence de Cortex sur le compte d'essai sans toucher au reste de l'architecture.

Le projet nous a aussi appris que les difficultés se trouvent souvent entre les outils plutôt que dans chacun d'eux : noms de tables normalisés par DLT, extension de fichier ignorée par Dagster, propriété des schémas dans Snowflake, variables d'environnement non interprétées. Aucun de ces problèmes n'était complexe, mais chacun bloquait la chaîne entière.

Perspectives

Plusieurs suites sont envisageables :

- utiliser Cortex sur un compte Snowflake payant et comparer ses requêtes à celles d'Ollama ;
- constituer un jeu d'évaluation d'une cinquantaine de questions avec leur réponse attendue, pour mesurer la qualité du Text-to-SQL ;
- créer un rôle Snowflake en lecture seule sur `GOLD` pour l'application ;
- historiser les dimensions clients et produits avec les *snapshots* de dbt ;
- étendre le modèle aux achats et aux stocks, présents dans la base source mais non chargés ;
- déployer l'application et Dagster sur un serveur, avec une authentification par cookie

plutôt que par l’adresse.

Annexes

A. Commandes principales

Les commandes suivantes reconstruisent la plateforme à partir du dépôt. Elles supposent SQL Server accessible et le fichier `.dlt/secrets.toml` renseigné.

```
# Installation
uv sync && source .venv/bin/activate

# Développement local (DuckDB)
export DUCKDB_PATH=$PWD/warehouse.duckdb
dg check defs                  # valide la configuration Dagster
dg list defs                   # liste les actifs, le job et la planification
dg dev                         # interface Dagster : http://localhost:3000

# Production (Snowflake)
export DLT_DESTINATION=snowflake DBT_TARGET=prod
export SF_ACCOUNT=<compte> SF_USER=<utilisateur> SF_PASSWORD='<mot_de_passe>'
dg dev                         # puis « Materialize all »

# Modèle local pour la génération du SQL
ollama pull qwen2.5-coder:7b

# Application
streamlit run presentation_app/app.py       # http://localhost:8501
```

B. Variables d'environnement

Variable	Valeur par défaut	Rôle
`DLT_DESTINATION`	`duckdb`	destination de l'ingestion
`DBT_TARGET`	`dev`	cible dbt (`dev` ou `prod`)
`DUCKDB_PATH`	`warehouse.duckdb`	fichier DuckDB local
`SF_ACCOUNT`	aucune	identifiant du compte Snowflake
`SF_USER`	aucune	utilisateur Snowflake
`SF_PASSWORD`	aucune	mot de passe Snowflake
`SF_ROLE`	`PFPB_ROLE`	rôle utilisé par l'application

Variable	Valeur par défaut	Rôle
`SF_WAREHOUSE`	`COMPUTE_WH`	warehouse Snowflake
`SQL_GENERATOR`	`auto`	`auto`, `cortex`, `ollama` ou `none`
`CORTEX_MODEL`	`mistral-large2`	modèle Cortex
`OLLAMA_MODEL`	`qwen2.5-coder:7b`	modèle Ollama
`OLLAMA_URL`	`http://localhost:11434`	adresse du serveur Ollama

C. Exécution d'une question dans l'application

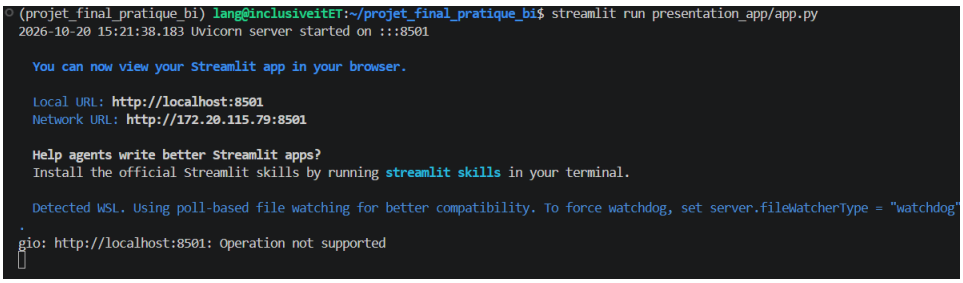
La fonction `_ask` du fichier `presentation_app/views/workspace.py` enchaîne les étapes décrites au chapitre 3.

```
def _ask(user: dict, folder_id: int, question: str, sql: str | None = None) -> None:
    st.html(f'<div class="wwi-question"><div>{html.escape(question)}</div></div>')
    start = time.perf_counter()
    engine = "manuel" if sql else None
    try:
        with st.spinner("Génération du SQL et exécution sur la couche …Gold"):
            if not sql:
                sql, engine = warehouse.generate_sql(question)
            sql = warehouse.clean_sql(sql)
            df = warehouse.run_sql(sql)
    except warehouse.QueryError as exc:
        st.session_state.last_error = (question, str(exc), sql)
        st.rerun()
    st.session_state.pop("last_error", None)
    users_db.add_analysis(user["id"], folder_id, question, sql,
                          df.to_json(orient="split", date_format="iso", index=False),
  len(df),
                          engine, int((time.perf_counter() - start) * 1000))
    st.rerun()
```

Une requête saisie directement par l'utilisateur est marquée `manuel` et passe par le même contrôle `clean_sql` que le SQL généré. Toute erreur, qu'elle vienne du modèle, du filtre ou de la base, est convertie en `QueryError` et affichée sous la question. En cas de succès, le résultat est enregistré avec le moteur utilisé et la durée totale en millisecondes.

D. Lancement de l'application

La commande `streamlit run presentation_app/app.py` démarre l'application sur le port 8501.


(projet_final_pratique_bi) lang@inclusiveitET:~/projet_final_pratique_bi$ streamlit run presentation_app/app.py
2026-10-20 15:21:38.183 Uvicorn server started on :::8501
You can now view your Streamlit app in your browser.
Local URL: http://localhost:8501
Network URL: http://172.20.115.79:8501
Help agents write better Streamlit apps?
Install the official Streamlit skills by running streamlit skills in your terminal.
Detected WSL. Using poll-based file watching for better compatibility. To force watchdog, set server.fileWatcherType = "watchdog"
gio: http://localhost:8501: Operation not supported